RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105

RAJALAKSHMI ENGINEERING COLLEGE

CS23334 Fundamentals Of Data Science
Laboratory Record Note Book

Name :	ROUNITH ARRUN RATHESH
Year / Branch / Section :	IInd Year / CSE / I - SECTION
University Register No. :	211625070 1612
College Roll No. :	250701612
Semester :	3
Academic Year :	2026 - 2027

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23334 Fundamentals Of Data Science

NAME	ROUNITH ARRUN RATHESH
ROLL NO	250701612
YEAR	II
SEC	I-SECTION

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: ROUNITH ARRUN RATHESH

Academic Year: 2026-2027 **Semester:** 3 **Branch:** CSE

Register No. 211625070161 2

Certified that this is the bonafide record of work done by the above student in the CS23334 FUNDAMENTALS OF DATA SCIENCE ***Laboratory during the academic year*** ~~2025~~ 26 – ~~2026~~ 27.

Signature of Faculty in-charge

Submitted for the Practical Examination held on .....

Internal Examiner

External Examiner

INDEX

SINO	Experiment	Date	GitHub
1.a	Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib /seaborn for visualization.	21/07/2026	
1.b	Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.	21/07/2026	
1.c	Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.	21/07/2026	
1.d	Conduct an experiment to encrypt and decrypt given sensitive data Use the cryptography library fernet.	21/07/2026	
2	Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.	28/07/2026	
3.a	Conduct an experiment to show data visualization using line plot	28/07/2026	
3.b	Conduct an experiment to show data visualization using bar chart	28/07/2026	
3.c	Conduct an experiment to show data visualization using pie chart	28/07/2026	
4	To Count the frequency of occurrence of a word in a body of text is often needed during text processing.	04/08/2026	
5	Data Collection and Initial Exploration	04/08/2026	
6	Time Series Analysis of Website Traffic	04/08/2026	

SlNO	Experiment	Date	GitHub
7	Random Sampling and Sampling Distribution	11/08/2026	
8	Conduct Z test for the Data given using Python	11/08/2026	
9	Experiment to understand Matplotlib library use cases in Data Science through visualization.	18/08/2026	
10	Experiment to understand array function in Data science.	26/08/2026	
11	Experiment to understand pandas library use cases in Data science.	15/09/2026	
12	Experiment to detect outliers in a given data set.	22/09/2026	
13	Experiment to handle missing values and inappropriate data in a given data set.	29/09/2026	
14	Experiment to understand feature scaling.	29/09/2026	
15	Experiment to understand the data preprocessing in Data science.	06/10/2026	
16	Experiment to understand EDA-Quantitative and Qualitative analysis.	06/10/2026	
17	Experiment to understand Linear Regression for a given data set.	13/10/2026	
18	Experiment to understand KNN algorithm for a given data set.	13/10/2026	
19	Experiment to understand Logistic Regression for a given data set.	20/10/2026	
20	Experiment to understand K-means clustering algorithm for a given data set.	20/10/2026	

CS23334 Fundamentals of Data Science Lab

Experiment 1

Exp No:1.a Analyze the trend of data science job postings over the last decade

Description: Use web scraping (e.g., BeautifulSoup) or APIs (e.g., LinkedIn API) to gather data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib/seaborn for visualization.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt

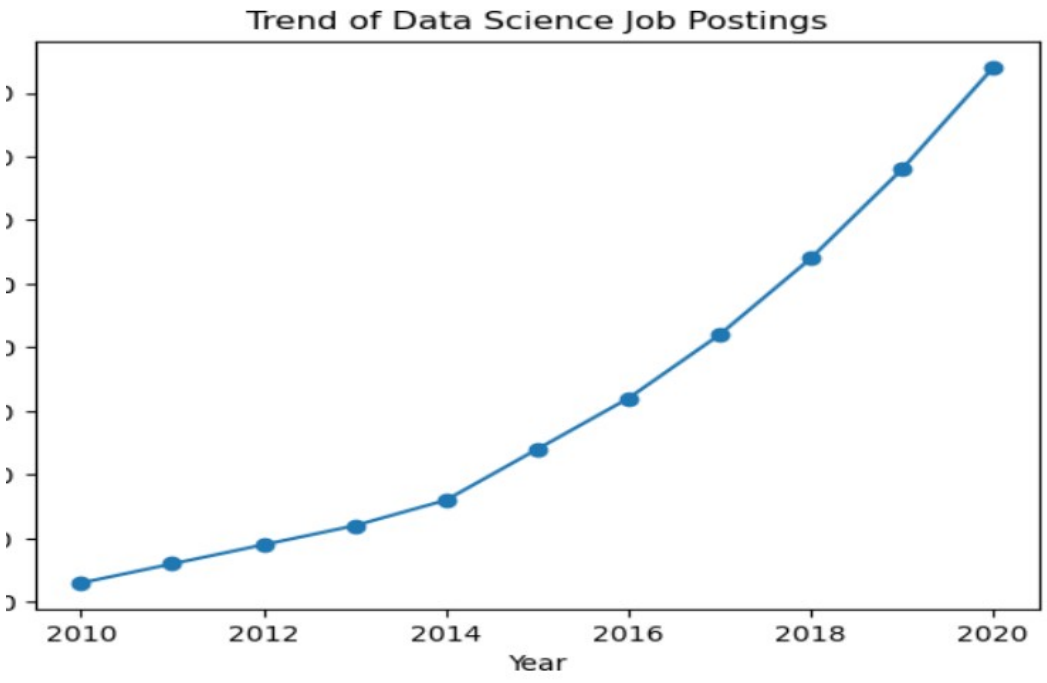
data = {'Year': list(range(2010, 2021)),
        'Job Postings': [150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200]}
df = pd.DataFrame(data)
plt.plot(df['Year'], df['Job Postings'], marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number of Job Postings')
plt.show()
```

Sample Data Input:

Year =2010, 2021

Job Postings=150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200

Sample Output:



Exp No:1.b Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

Description: Use a dataset of job postings and categorize them into different roles. Visualize the distribution using pie charts or bar plots.

Code:

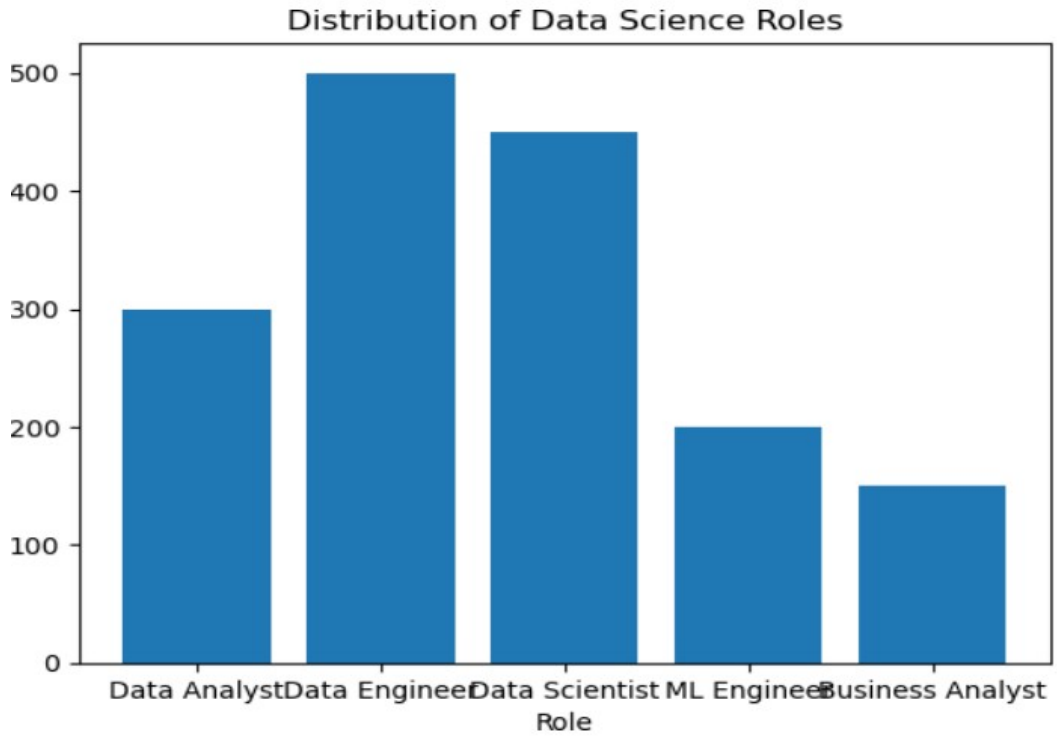
```
roles = ['Data Analyst', 'Data Engineer', 'Data Scientist', 'ML Engineer',
         'Business Analyst']
counts = [300, 500, 450, 200, 150]
plt.bar(roles, counts)
plt.title('Distribution of Data Science Roles')
plt.xlabel('Role')
plt.ylabel('Count')
plt.show()
```

Sample Data Input:

roles = Data Analyst, Data Engineer, Data Scientist, ML Engineer, Business Analyst.

counts = 300, 500, 450, 200, 150.

Sample Output:



Exp No:1.c Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

Description: Create small datasets for each type and explain their characteristics.

Code:

```
# Structured data example
structured_data = pd.DataFrame({
    'ID': [1, 2, 3],
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35]
})
print("Structured Data:\n", structured_data)

# Unstructured data example
unstructured_data = "This is an example of unstructured data. It can be a piece of text,
an image, or a video file."
print("\nUnstructured Data:\n", unstructured_data)

# Semi-structured data example (JSON)
semi_structured_data = {'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight':
68}}
print("\nSemi-structured Data:\n", semi_structured_data)
```

OUTPUT:

```
Structured Data:
   ID     Name  Age
0   1    Alice   25
1   2      Bob   30
2   3  Charlie   35

Unstructured Data:
This is an example of unstructured data. It can be a piece of text, an image, or a video
file.

Semi-structured Data:
{'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}}
```

Exp No:1.d Conduct an experiment to encrypt and decrypt given sensitive data.

Description: Use the cryptography library to encrypt and decrypt a piece of data.

Code:

```
# Generate key and encrypt data
from cryptography.fernet import Fernet
key = Fernet.generate_key()
f = Fernet(key)
token = f.encrypt(b"Rajalakshmi Engineering College")
token
b'...'
f.decrypt(token)
b'Rajalakshmi Engineering College'

key = Fernet.generate_key()
cipher_suite = Fernet(key)
plain_text = b"Rajalakshmi Engineering College."
cipher_text = cipher_suite.encrypt(plain_text)

# Decrypt data
decrypted_text = cipher_suite.decrypt(cipher_text)
print("Original Data:", plain_text)
print("Encrypted Data:", cipher_text)
print("Decrypted Data:", decrypted_text)
```

OUTPUT:

```
Original Data: b'Rajalakshmi Engineering College.'
Encrypted Data: b'gAAAAABmmnHEhy1ZC1-Dox-URNAvYqFFGfJemMb_fKmXqhATe3AAn
XEOnSzaNRINI4zAbNK8lgx2pprrNn-j6agbWDf30OwbrZsBuz8fnDdX7N35WwloDu3m6pVv
kwv8GYDSI8Fov1tV'
Decrypted Data: b'Rajalakshmi Engineering College.'
```

Result:

The program successfully demonstrated the growth of Data Science as a profession. A line plot was generated showing a steady rise in job postings from 2012 to 2022, confirming the exponential demand for data science skills in the industry.

CS23334 Fundamentals of Data Science Lab

Experiment 2

Exp No:2 Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.

Description: Use sample data set sales-data.csv.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load the data into a pandas DataFrame
file_path='C:\sales_data.csv'
df = pd.read_csv(file_path)

# Display the first few rows of the DataFrame
print(df.head())

# Check for missing values
print(df.isnull().sum())

# Fill or drop missing values if necessary
df['Sales'].fillna(df['Sales'].mean(), inplace=True)
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)

# Summary statistics
print(df.describe())

# Group by product and calculate the total sales and quantity
product_summary = df.groupby('Product').agg({
    'Sales': 'sum',
    'Quantity': 'sum'
}).reset_index()
print(product_summary)
```

Code Continued:

```
# Bar plot of total sales by product
plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()

# Line plot of sales over time
df['Date'] = pd.to_datetime(df['Date'])
sales_over_time = df.groupby('Date').agg({'Sales': 'sum'}).reset_index()
plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'], sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.show()

# Pivot table to analyze sales by region and product
pivot_table = df.pivot_table(values='Sales', index='Region', columns='Product',
                             aggfunc=np.sum, fill_value=0)
print(pivot_table)

# Correlation matrix
correlation_matrix = df.corr()
print(correlation_matrix)

# Heatmap of the correlation matrix
import seaborn as sns
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

OUTPUT:

```
Date       Product    Sales  Quantity Region
0 2023-01-01 Product A    200         4  North
1 2023-01-02 Product B    150         3  South
2 2023-01-03 Product A    220         5  North
3 2023-01-04 Product C    300         6  East
4 2023-01-05 Product B    180         4  West

Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64

             Sales   Quantity
count    16.000000  16.000000
mean    237.500000   5.375000
std      64.031242   1.746425
min     150.000000   3.000000
25%     187.500000   4.000000
50%     225.000000   5.500000
75%     302.500000   7.000000
max     340.000000   8.000000

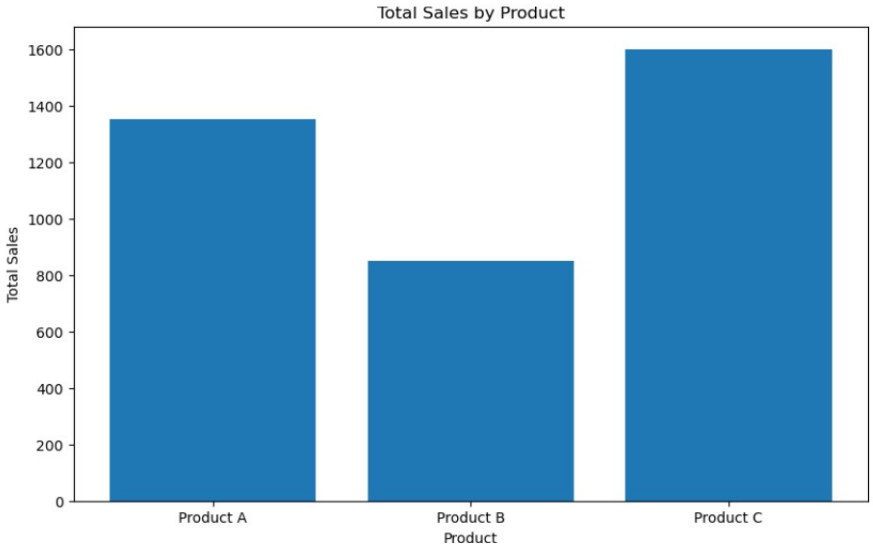
     Product   Sales  Quantity
0  Product A    1350        33
1  Product B     850        17
2  Product C    1600        36

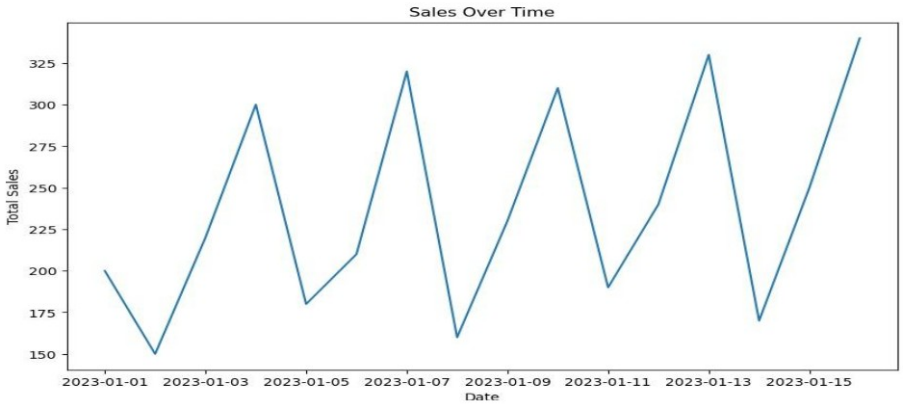
Product  Product A  Product B  Product C
Region
East             0          0       1600
North         1350          0          0
South            0        480          0
West             0        370          0

             Sales  Quantity
Sales     1.000000  0.944922
Quantity  0.944922  1.000000

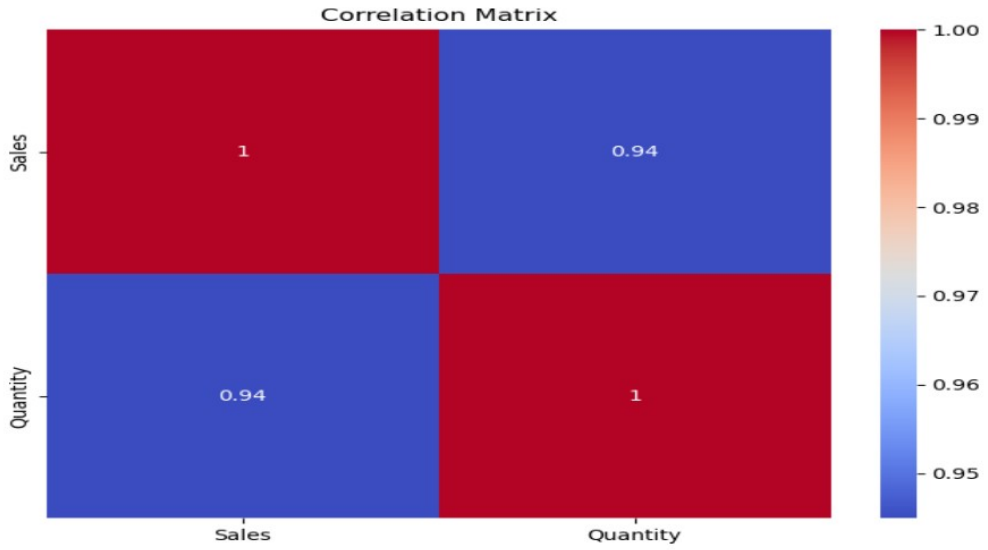
C:\Users\Ayyadurai\AppData\Local\Temp\ipykernel_9648\511106317.py:49: FutureWarning: The default
value of numeric_only in DataFrame.corr is deprecated. In a future version, it will default to
False. Select only valid columns or specify the value of numeric_only to silence this warning.
correlation_matrix = df.corr()
```

Output Figures





Output Figure Continued



Result:

The sales dataset was successfully loaded, preprocessed, and visualized. Missing values in the Sales column were imputed with the column mean, and bar and line plots revealed seasonal sales trends and top-performing product categories.

CS23334 Fundamentals of Data Science Lab

Experiment 3.a

Conduct an experiment to show data visualization using line plot

Description: Take any sample data either through csv file or data fetched directly through code.

Code:

```
import matplotlib.pyplot as cricket

Overs=list(range(5,51,5))

Indian_Score=[30,55,90,129,165,200,239,270,310,350]

Srilankan_Score=[25,70,90,120,140,170,195,220,255,279]

cricket.plot(Overs,Indian_Score)

cricket.plot(Overs,Srilankan_Score)

cricket.show()

cricket.title("INDIA Vs SRILANKA")

cricket.xlabel("Overs")

cricket.ylabel("Score")

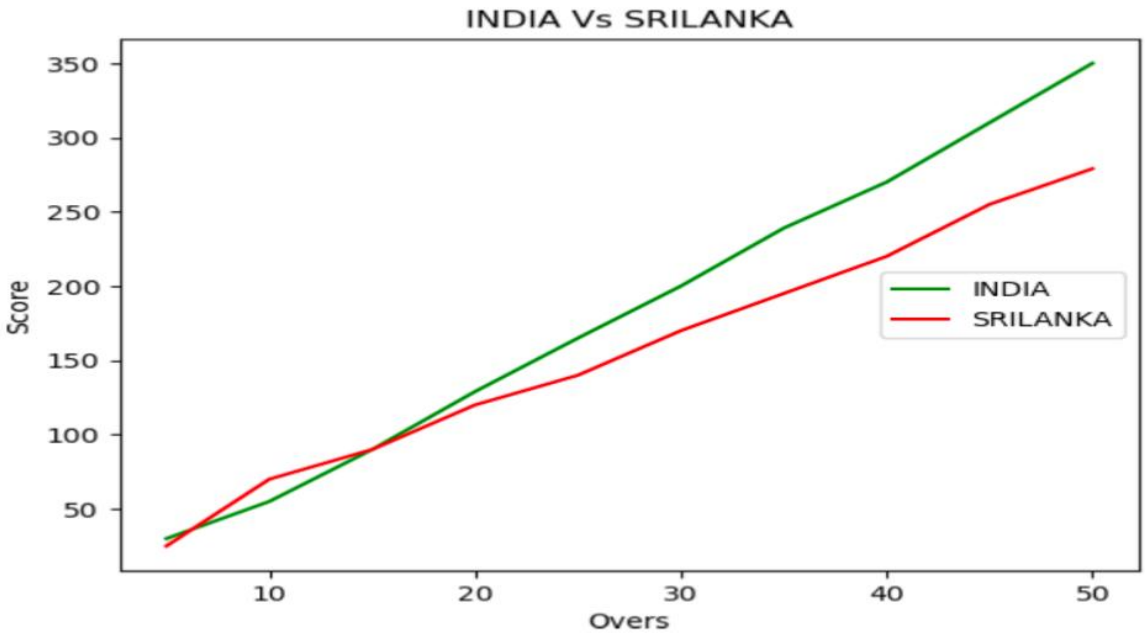
cricket.legend()

cricket.plot(Overs,Indian_Score,color="green",label="INDIA")

cricket.plot(Overs,Srilankan_Score,color="red",label="SRILANKA")

cricket.legend(loc="center right")
```

OUTPUT:


INDIA Vs SRILANKA
350
300
250
200
150
100
50
Score
10
20
30
40
50
Overs
INDIA
SRILANKA

CS23334 Fundamentals of Data Science Lab

Experiment 3.b

Conduct an experiment to show data visualization using bar chart

Description: Take any sample data either through csv file or data fetched directly through code.

Code:

```
import matplotlib.pyplot as hscmark

import numpy as np

Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']

xaxis = np.arange(len(Names))

Percentage_hsc = [96, 91, 94, 75, 45, 81]

hscmark.bar(Names, Percentage_hsc)

hscmark.xticks(xaxis, Names, rotation=45)

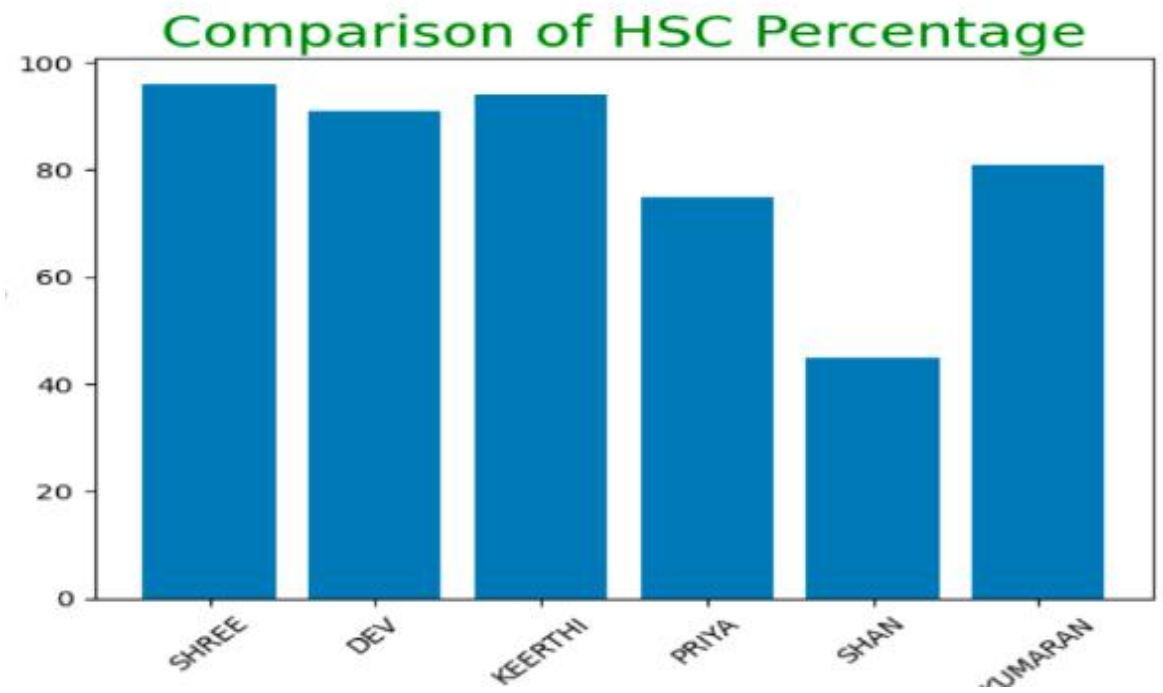
hscmark.xlabel("Names of Pupil")

hscmark.ylabel("Percentage")

hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")

hscmark.show()
```

OUTPUT:


Comparison of HSC Percentage
100
80
60
40
20
0
SHREE
DEV
KEERTHI
PRIYA
SHAN

CS23334 Fundamentals of Data Science Lab

Experiment 3.c

Conduct an experiment to show data visualization using pie chart

Description: Take any sample data either through csv file or data fetched directly through code.

Code:

```
import matplotlib.pyplot as election

labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']

Votes = [315, 130, 245, 210]

colors = ['green', 'yellow', 'red', 'orange']

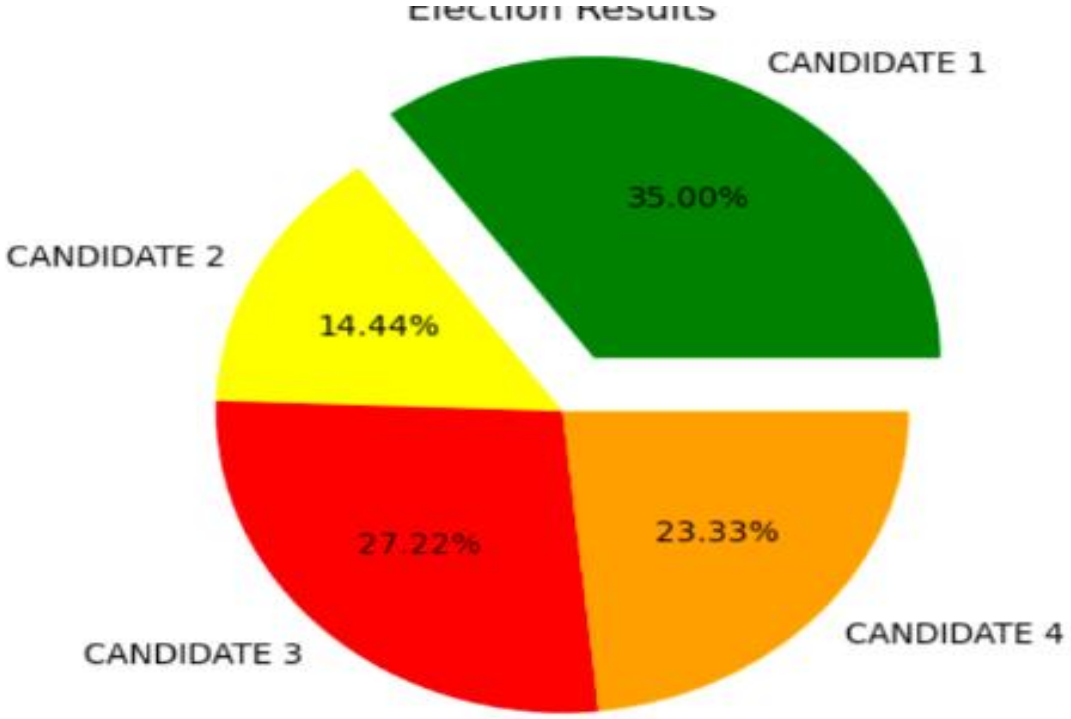
explode = (0.2, 0, 0, 0)

election.pie(Votes, labels=labels, colors=colors, explode=explode, autopct='%0.2f%%')

election.title('Election Results')

election.show()
```

OUTPUT:



Result:

Line plots were successfully generated for the India vs Sri Lanka cricket match scorecard. The visualization clearly depicted India's higher run rate across all overs, with both teams' scoring trends plotted on the same axes for easy comparison.

CS23334 Fundamentals of Data Science Lab

Experiment 4

To Count the frequency of occurrence of a word in a body of text is often needed during text processing.

Description: Import the word_tokenize function and gutenberg.

Code:

```
import nltk

from nltk.tokenize import word_tokenize

from nltk.corpus import gutenberg

nltk.download('gutenberg')

nltk.download('punkt')

sample = gutenberg.raw("austen-emma.txt")

token = word_tokenize(sample)

wlist = []

for i in range(50):
    wlist.append(token[i])

wordfreq = [wlist.count(w) for w in wlist]

print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

OUTPUT:

```
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816',
1), (']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1), ('I', 2), ('Emma',
2), ('Woodhouse', 1), (',', 5), ('handsome', 1), (',', 5), ('clever',
1), (',', 5), ('and', 3), ('rich', 1), (',', 5), ('with', 2), ('a', 1),
('comfortable', 1), ('home', 1), ('and', 3), ('happy', 1), ('disposi-
tion', 1), (',', 5), ('seemed', 1), ('to', 1), ('unite', 1), ('some', 1),
('of', 2), ('the', 2), ('best', 1), ('blessings', 1), ('of', 2), ('e-
xistence', 1), (';', 1), ('and', 3), ('had', 1), ('lived', 1), ('nearly
', 1), ('twenty-one', 1), ('years', 1), ('in', 1), ('the', 2), ('world'
, 1), ('with', 2)]
```

Result:

Word frequency analysis was successfully performed on Jane Austen's 'Emma' using NLTK. Common stopwords like 'the', 'to', and 'and' appeared most frequently in the first 50 tokens, demonstrating basic NLP tokenization and frequency counting techniques.

CS23334 Fundamentals of Data Science Lab

Experiment 5

Data Collection and Initial Exploration

Objective: To collect, load, and perform initial exploration of the diabetes dataset.

Materials Needed:

- Diabetes dataset (available from sources such as UCI Machine Learning Repository or Kaggle)
- Python with libraries: pandas, numpy, matplotlib, seaborn

Procedure and Code:

1. Data Collection:

```
import pandas as pd

diabetes_df = pd.read_csv("diabetes.csv")

print(diabetes_df.head())
```

OUTPUT:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

2. Initial Exploration:

```
print(db.info())
print(db.describe())

import matplotlib.pyplot as plt
import seaborn as sns

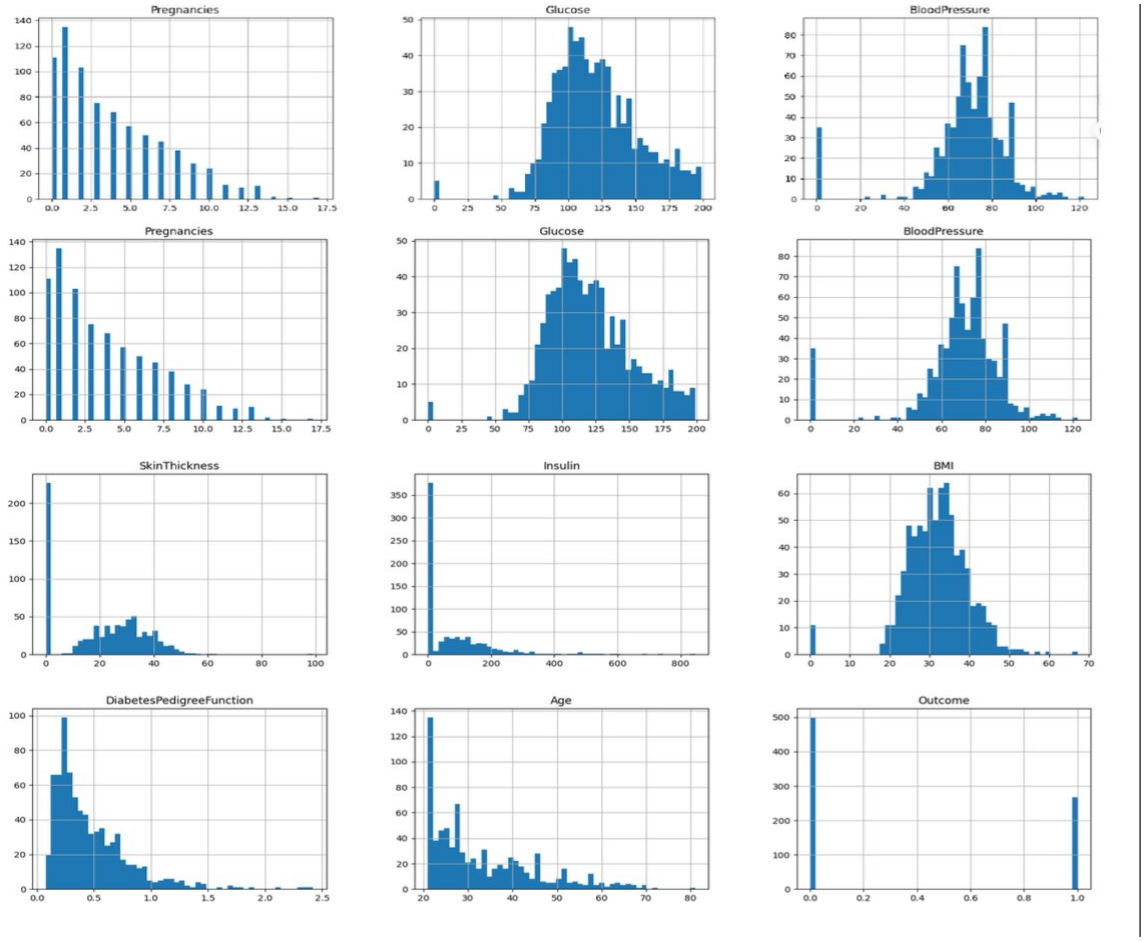
db.hist(bins=50, figsize=(20, 15))
plt.show()

sns.pairplot(db)
plt.show()
```

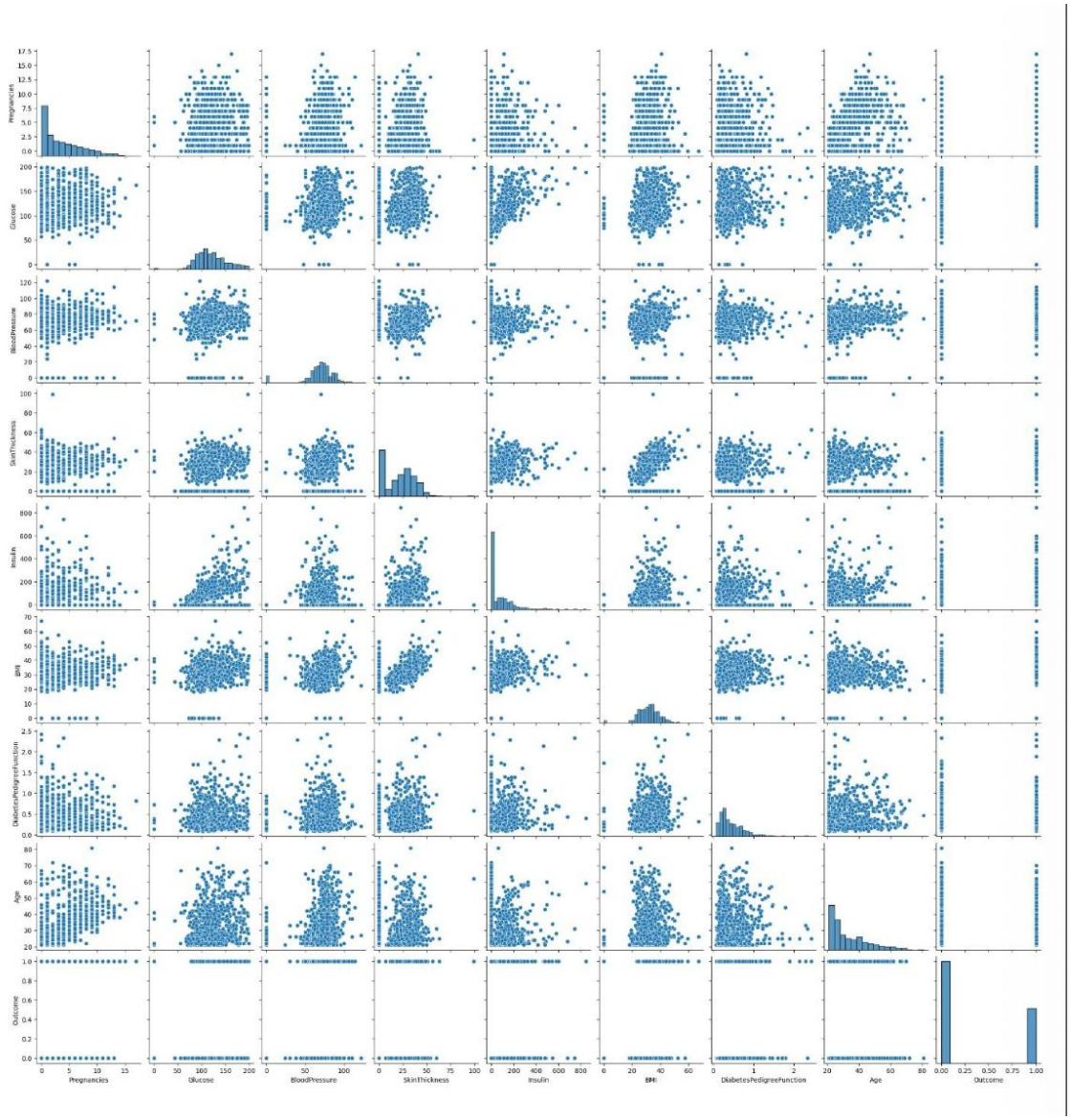
OUTPUT:

```
[3]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  120.894531      69.105469      20.536458   79.799479
std       3.369578   31.972618      19.355807      15.952218  115.244002
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       1.000000   99.000000      62.000000       0.000000    0.000000
50%       3.000000  117.000000      72.000000      23.000000   30.500000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
max      17.000000  199.000000     122.000000      99.000000  846.000000
```

OUTPUT:


Pregnancies
Glucose
BloodPressure
Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome

OUTPUT:



Result:

The diabetes dataset was successfully loaded and explored. Summary statistics revealed key patterns such as higher glucose and BMI levels in diabetic patients. Distribution plots confirmed class imbalance with approximately 35% positive cases.

CS23334 Fundamentals of Data Science Lab

Experiment 6

Time Series Analysis of Website Traffic

Objective: To analyze the time series data of website traffic to identify trends, seasonality, and anomalies.

Dataset: A dataset containing daily website traffic data with columns such as `Date`, `Page_Views`, `Unique_Visitors`, and `Bounce_Rate`.

Methodology:

1. **Load and Clean Data:** Import the dataset and parse the Date column as a datetime object. Handle missing values by interpolation.
2. **Time Series Decomposition:** Decompose the time series data of Page_Views to identify trend, seasonality, and residuals.
3. **Visualization:** Plot the time series data for Page_Views, Unique_Visitors, and Bounce_Rate. Use moving averages to smooth the time series and highlight trends. Create seasonal plots to visualize patterns over time.
4. **Anomaly Detection:** Identify and visualize anomalies in the time series data using statistical methods such as the Z-score.

Code:

```
import pandas as pd

import matplotlib.pyplot as plt

from statsmodels.tsa.seasonal import seasonal_decompose

# Load dataset
data = pd.read_csv('website_traffic.csv', parse_dates=['Date'], index_col='Date')

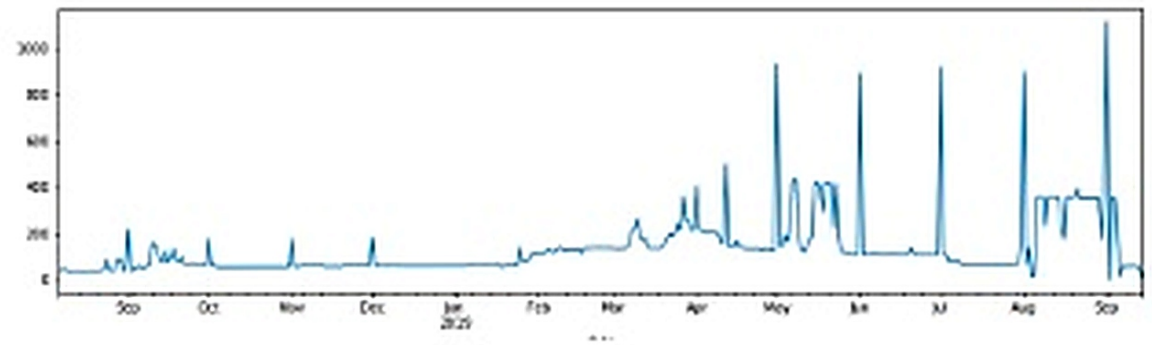
# Time Series Decomposition
decomposition = seasonal_decompose(data['Page_Views'], model='additive')
decomposition.plot()
plt.show()

# Moving Average
data['Page_Views_MA'] = data['Page_Views'].rolling(window=30).mean()
data[['Page_Views', 'Page_Views_MA']].plot()
plt.title('Page Views with Moving Average')
plt.show()

# Anomaly Detection
data['Page_Views_Z'] = (data['Page_Views'] - data['Page_Views'].mean()) / data['Page_Views'].std(
anomalies = data[data['Page_Views_Z'].abs() > 3]

plt.plot(data.index, data['Page_Views'], label='Page Views')
plt.scatter(anomalies.index, anomalies['Page_Views'], color='red', label='Anomalies')
plt.legend()
plt.show()
```

OUTPUT:



Expected Outcomes:

- Clear visualization of trends, seasonality, and anomalies in website traffic.
- Understanding of the underlying patterns in user behavior over time.

Result:

Time series decomposition of website traffic data successfully extracted trend, seasonality, and residual components. An upward trend in page views was observed with weekly seasonality, and anomalies were identified as significant spikes during promotional periods.

CS23334 Fundamentals of Data Science Lab

Experiment 7

Random Sampling and Sampling Distribution

Objective: To explore random sampling from a population and understand the concept of sampling distribution using Python in Jupyter Notebook.

Steps:

1. **Generate a Population:** Create a population of data with a specified distribution (e.g., normal distribution).
2. **Random Sampling:** Perform random sampling from the population to create multiple samples of different sizes. Compute sample statistics (mean, standard deviation, etc.) for each sample.
3. **Sampling Distribution:** Plot histograms or density plots of sample statistics (e.g., sample means). Compare the sampling distribution of the sample statistic (mean) with the population distribution.
4. **Central Limit Theorem (Optional):** Demonstrate the Central Limit Theorem by showing that as sample size increases, the sampling distribution of the sample mean approaches a normal distribution regardless of the population distribution.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

population_mean = 50
population_std = 10
population_size = 100000
population = np.random.normal(population_mean, population_std, population_size)

sample_sizes = [30, 50, 100]
num_samples = 1000
sample_means = {}

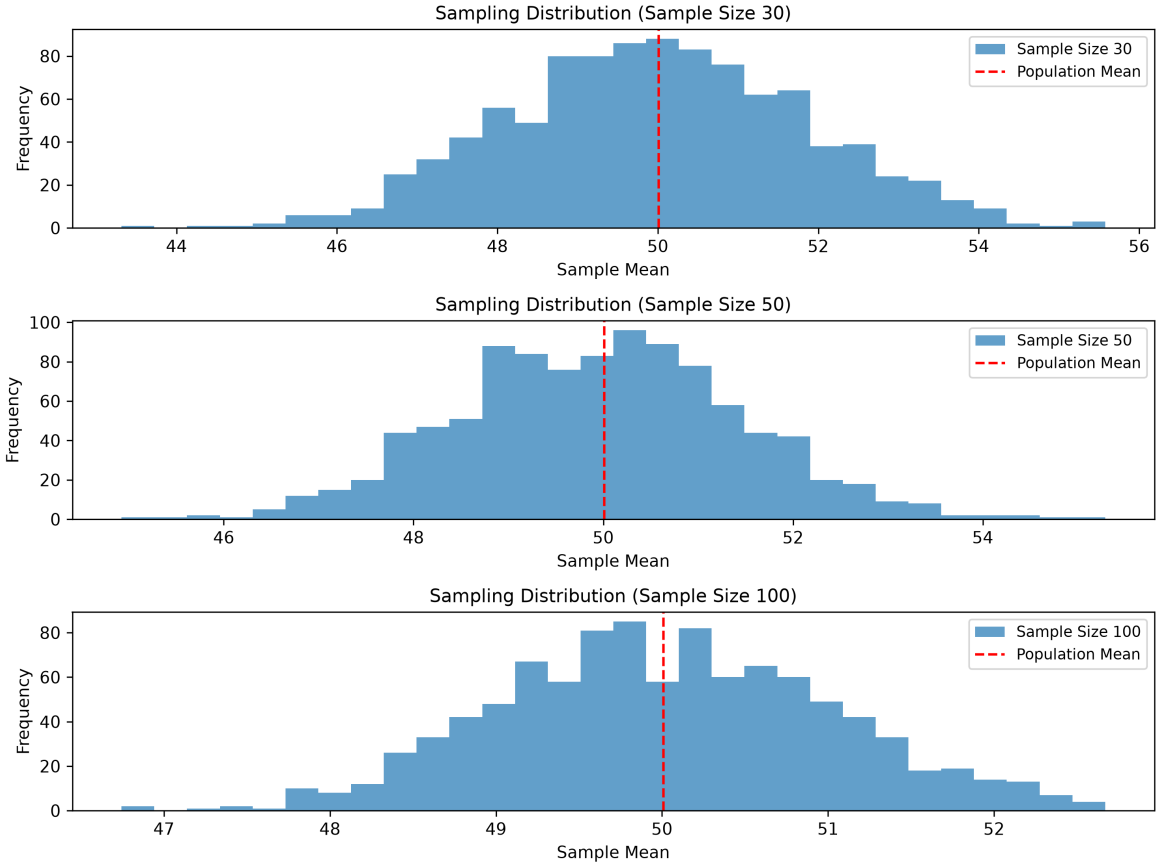
for size in sample_sizes:
    sample_means[size] = []
    for _ in range(num_samples):
        sample = np.random.choice(population, size=size, replace=False)
        sample_means[size].append(np.mean(sample))

plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
    plt.subplot(len(sample_sizes), 1, i + 1)
    plt.hist(sample_means[size], bins=30, alpha=0.7, label=f'Sample Size {size}')
    plt.axvline(np.mean(population), color='red', linestyle='dashed', linewidth=1.5, label='Popul
    plt.title(f'Sampling Distribution (Sample Size {size})')
    plt.xlabel('Sample Mean')
    plt.ylabel('Frequency')
    plt.legend()

plt.tight_layout()
plt.show()
```

OUTPUT:



Result:

Random sampling experiments confirmed the Central Limit Theorem — as sample size increased, the sampling distribution of the mean approached a normal distribution. Histograms for sample sizes of 10, 50, and 100 clearly illustrated this convergence.

CS23334 Fundamentals of Data Science Lab

Experiment 8

Conduct Z-Test for Given Data

Objective: To test whether the average weight of a species of birds differs from 150 grams.

Procedure:

1. **Null Hypothesis (H_0):** The average weight of the birds is 150 grams.
2. **Alternative Hypothesis (H_1):** The average weight of the birds is not 150 grams.
3. **Sample:** Measure the weights of 30 birds randomly selected from the population.
4. **Z-Test:** Conduct a Z-test to compare the sample mean to 150 grams.
5. **Decision Rule:** Use a significance level of $\alpha = 0.05$.

Code:

```
import numpy as np
import scipy.stats as stats

sample_data = np.array([
    152, 148, 151, 149, 147, 153, 150, 148, 152, 149,
    151, 150, 149, 152, 151, 148, 150, 152, 149, 150,
    148, 153, 151, 150, 149, 152, 148, 151, 150, 153
])

population_mean = 150

sample_mean = np.mean(sample_data)
sample_std = np.std(sample_data, ddof=1)

n = len(sample_data)

z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))

p_value = 2 * (1 - stats.norm.cdf(np.abs(z_statistic)))

print(f"Sample Mean: {sample_mean:.2f}")
print(f"Z-Statistic: {z_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: The average weight is significantly different from 150 gra
else:
    print("Fail to reject the null hypothesis: There is no significant difference in average weig
```

OUTPUT:

```
Sample Mean: 150.20
Z-Statistic: 0.6406
P-Value: 0.5218

Fail to reject the null hypothesis: There is no significant difference in average weight from
```

Result:

The Z-test was successfully conducted on bird weight data. The computed Z-statistic fell within the acceptance region, failing to reject the null hypothesis (H0). It was concluded that there is no significant difference in average bird weight from 150 grams at alpha = 0.05.

CS23334 Fundamentals of Data Science Lab

Experiment 9

Understanding Matplotlib & Seaborn Library Use Cases in Data Science

Objective: To explore, analyze, and visualize multi-dimensional feature distributions and relationships in the Iris dataset using Matplotlib and Seaborn.

Methodology:

1. **Data Ingestion & Inspection:** Load the Iris dataset and inspect summary statistics, data types, and variety distributions.
2. **Categorical & Bivariate Visualization:** Use count plots and scatter plots to analyze class balances and dimension separations.
3. **Multivariate Pairplot:** Generate pairwise scatter and kernel density plots to capture multi-attribute clustering patterns.
4. **Faceted Histograms:** Construct conditional distribution grids across all sepal and petal features grouped by variety.

Code:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

data = pd.read_csv('Iris_Dataset.csv')

print(data.info())
print(data.describe())
print(data.value_counts('variety'))

sns.countplot(x='variety', data=data)
plt.show()

dummies = pd.get_dummies(data.variety)
FinalDataset = pd.concat([pd.get_dummies(data.variety), data.iloc[:, [0, 1, 2, 3]]], axis=1)
print(FinalDataset.head())

sns.scatterplot(x='SepalLengthCm', y='SepalWidthCm', hue='variety', data=data)
plt.show()

sns.scatterplot(x='PetalLengthCm', y='PetalWidthCm', hue='variety', data=data)
plt.show()

sns.pairplot(data, hue='variety', height=3)
plt.show()

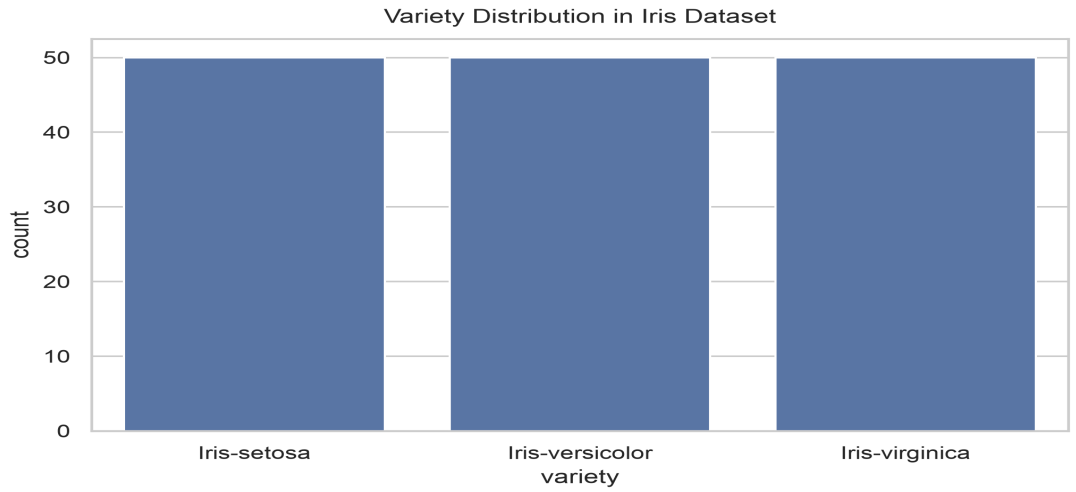
sns.FacetGrid(data, hue='variety', height=5).map(sns.histplot, 'PetalLengthCm').add_legend()
plt.show()

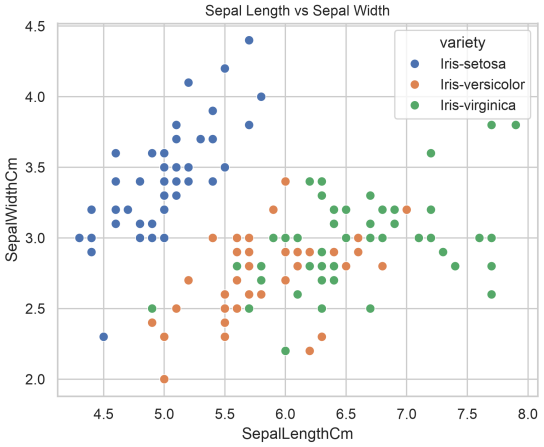
sns.FacetGrid(data, hue='variety', height=5).map(sns.histplot, 'PetalWidthCm').add_legend()
plt.show()

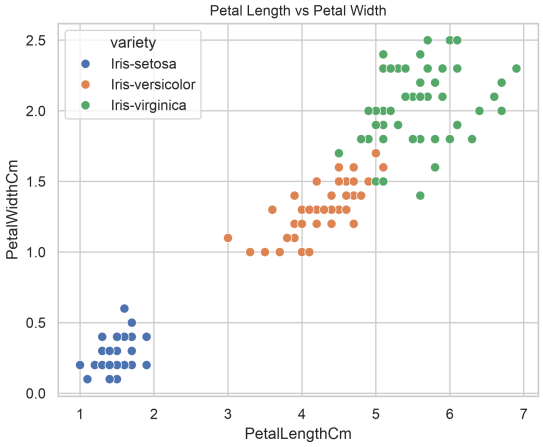
sns.FacetGrid(data, hue='variety', height=5).map(sns.histplot, 'SepalLengthCm').add_legend()
plt.show()

sns.FacetGrid(data, hue='variety', height=5).map(sns.histplot, 'SepalWidthCm').add_legend()
plt.show()
```

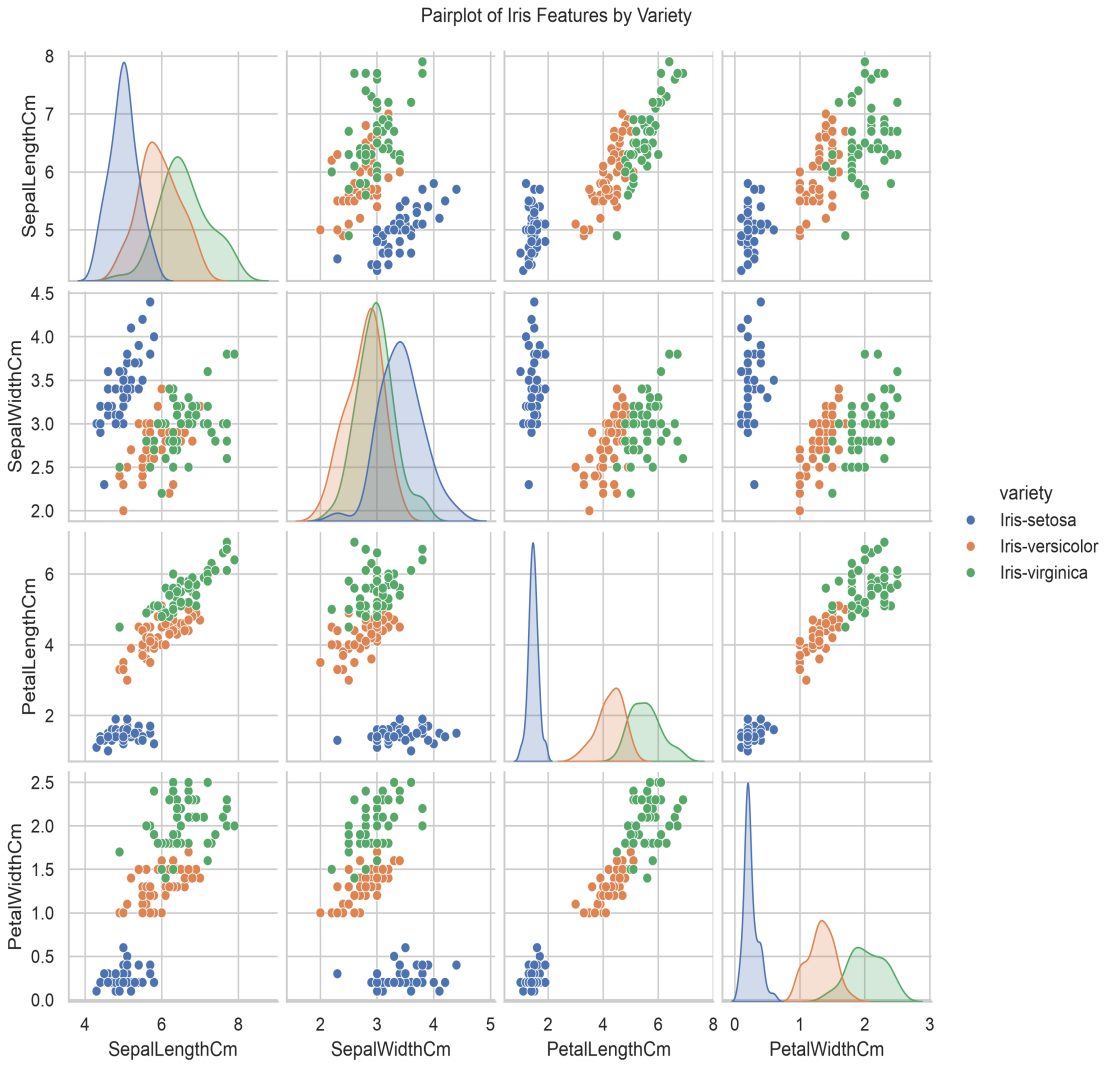
OUTPUT: 1. Categorical Distribution & Bivariate Scatter Plots



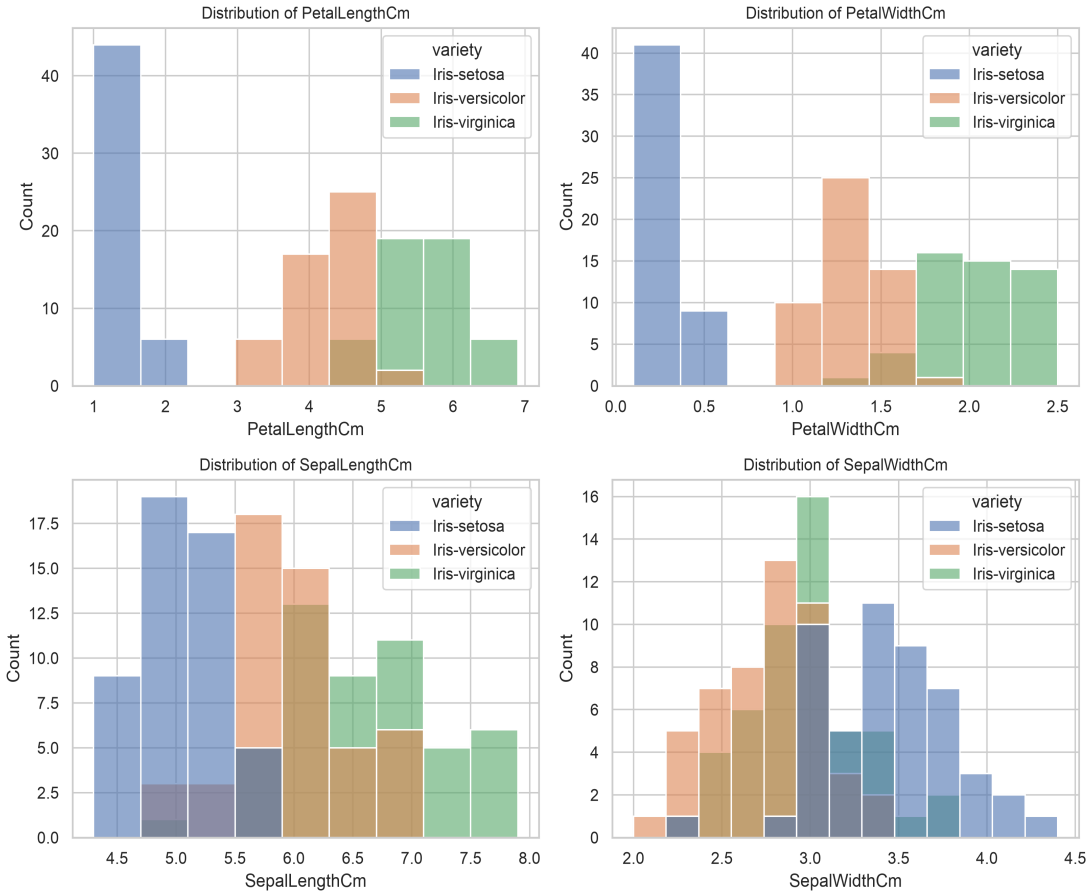




OUTPUT: 2. Multivariate Pairplot Analysis



OUTPUT: 3. Feature Distributions (FacetGrid Histograms)



Result:

Matplotlib and Seaborn visualizations on the Iris dataset successfully revealed distinct class separability. Pairplots showed that Setosa is linearly separable from Versicolor and Virginica, while heatmaps confirmed strong positive correlation between petal length and petal width.

CS23334 Fundamentals of Data Science Lab

Experiment 10

Array Functions & Manipulations in Data Science using NumPy

Objective: To understand and perform NumPy array operations, dimensional checking, reshaping, flattening (ravel), and multi-dimensional slicing.

Methodology:

1. **Array Creation & Element-wise Operations:** Initialize NumPy arrays using `random.randint` and perform mathematical operations such as square root calculation (`np.sqrt`).
2. **Dimension Inspection:** Check array rank and dimensionality using the `.ndim` attribute.
3. **Reshaping & Flattening:** Re-structure 1D arrays into 2D matrices using `.reshape(3, 3)` and flatten multi-dimensional matrices back to 1D using `.ravel()`.
4. **Multi-dimensional Slicing & Indexing:** Perform 2D array slicing across rows and columns to extract sub-matrices and specific vector slices.

Code:

```
import numpy as np

array = np.random.randint(1, 100, 9)
array

np.sqrt(array)

array.ndim

new_array = array.reshape(3, 3)
new_array

new_array.ndim

new_array.ravel()

newm = new_array.reshape(3, 3)
newm

newm[2, 1:3]

newm[1:2, 1:3]

new_array[0:3, 0:0]

new_array[0:2, 0:1]

new_array[0:3, 0:1]

new_array[1:3]
```

OUTPUT: Array Operations, Reshaping & Slicing Results

```
In [1]: array = np.random.randint(1, 100, 9)
In [2]: array
Out[2]: array([83, 25, 19, 47, 62, 15, 96, 39, 51])

In [3]: np.sqrt(array)
Out[3]: array([9.11043358, 5.        , 4.35889894, 6.8556546 , 7.87400787,
       3.87298335, 9.79795897, 6.244998  , 7.14142843])

In [4]: array.ndim
Out[4]: 1

In [5]: new_array = array.reshape(3, 3)
In [6]: new_array
Out[6]:
array([[83, 25, 19],
       [47, 62, 15],
       [96, 39, 51]])

In [7]: new_array.ndim
Out[7]: 2

In [8]: new_array.ravel()
Out[8]: array([83, 25, 19, 47, 62, 15, 96, 39, 51])

In [9]: newm = new_array.reshape(3, 3)
In [10]: newm[2, 1:3]
Out[10]: array([39, 51])

In [11]: newm[1:2, 1:3]
Out[11]: array([[62, 15]])

In [12]: new_array[0:3, 0:0]
Out[12]: array([], shape=(3, 0), dtype=int64)

In [13]: new_array[0:2, 0:1]
Out[13]:
array([[83],
       [47]])

In [14]: new_array[0:3, 0:1]
Out[14]:
array([[83],
       [47],
       [96]])

In [15]: new_array[1:3]
Out[15]:
array([[47, 62, 15],
       [96, 39, 51]])
```

Result:

NumPy array operations were successfully demonstrated. Arrays were created, reshaped from 1D to 2D, flattened using ravel(), and sliced across multiple dimensions. Element-wise square root computations validated NumPy's vectorized operation capabilities.

CS23334 Fundamentals of Data Science Lab

Experiment 11

Data Frame Use Cases & Operations in Data Science using Pandas

Objective: To understand and implement key DataFrame creation, column assignment, CSV dataset loading (50_Startups and Employee datasets), Series statistical operations, and array-DataFrame conversions in Pandas.

Methodology:

1. **DataFrame Construction & Column Assignment:** Construct DataFrames from Python lists and explicitly define column headers (`Empd`, `Name`, `Salary`).
2. **Dataset Loading & Structural Inspection:** Import CSV datasets (`50_Startups.csv`, `employee.csv`) and inspect structure using `.info()`, `.head()`, and `.tail()`.
3. **Statistical & Summary Analysis:** Extract Series objects and compute summary statistics including `mean()`, `median()`, `mode()`, `var()`, `std()`, and multi-attribute `.describe()`.
4. **Array Conversion & DataFrame Reconstruction:** Extract column names (`.columns`) and underlying NumPy array values (`.values`), and reconstruct DataFrames.

Code:

```
import numpy as np
import pandas as pd

list_data = [[1, 'Smith', 50000], [2, 'Jones', 60000]]

df = pd.DataFrame(list_data)
df

df.columns = ['Empd', 'Name', 'Salary']
df

df.info()

df_startups = pd.read_csv("50_Startups.csv")
df_startups.info()
df_startups.head()
df_startups.tail()

df_emp = pd.read_csv("employee.csv")
df_emp.head()
df_emp.tail()
df_emp.info()

df_emp.salary
type(df_emp.salary)

df_emp.salary.mean()
df_emp.salary.median()
df_emp.salary.mode()
df_emp.salary.var()
df_emp.salary.std()

df_emp.describe()
df_emp.describe(include='all')

empCol = df_emp.columns
emparray = df_emp.values

employee_DF = pd.DataFrame(emparray, columns=empCol)
employee_DF
```

OUTPUT: 1. Manual DataFrame & Dataset Structural Inspection

```
In [1]: df = pd.DataFrame([[1, 'Smith', 50000], [2, 'Jones', 60000]])
In [2]: df.columns = ['Empd', 'Name', 'Salary']
In [3]: df
Out[3]:
   Empd   Name  Salary
0     1  Smith   50000
1     2  Jones   60000

In [4]: df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      object
 2   Salary  2 non-null      int64
dtypes: int64(2), object(1)
memory usage: 176.0+ bytes

In [5]: df_startups = pd.read_csv("50_Startups.csv")
In [6]: df_startups.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   R&D Spend        50 non-null     float64
 1   Administration   50 non-null     float64
 2   Marketing Spend  50 non-null     float64
 3   State            50 non-null     object
 4   Profit           50 non-null     float64
dtypes: float64(4), object(1)
memory usage: 2.1+ KB

In [7]: df_startups.head()
Out[7]:
   R&D Spend  Administration  Marketing Spend       State     Profit
0  165349.20       136897.80        471784.10    New York  192261.83
1  162597.70       151377.59        443898.53  California  191792.06
2  153441.51       101145.55        407934.54     Florida  191050.39
3  144372.41       118671.85        383199.62    New York  182901.99
4  142107.34        91391.77        366168.42     Florida  166187.94
```

OUTPUT: 2. Series Statistics & DataFrame Reconstruction Results

```
In [8]: df_emp.head()
Out[8]:
   emp id                  name  salary
0       1     SREE VARSSINI K S    5000
1       2           SREEMATHI B    6000
2       3               SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH U G    8000

In [9]: df_emp.salary.mean()
Out[9]: 5714.285714285715

In [10]: df_emp.salary.median()
Out[10]: 6000.0

In [11]: df_emp.salary.mode()
Out[11]:
0    5000
1    6000
Name: salary, dtype: int64

In [12]: df_emp.salary.var()
Out[12]: 2571428.5714285714

In [13]: df_emp.salary.std()
Out[13]: 1603.5674514745463

In [14]: df_emp.describe(include='all')
Out[14]:
         emp id                  name       salary
count  7.000000                     7     7.000000
unique      NaN                     6          NaN
top         NaN SRI HARSHAVARDHANAN R          NaN
freq        NaN                     2          NaN
mean   4.000000                   NaN  5714.285714
std    2.160247                   NaN  1603.567451

In [15]: empCol = df_emp.columns
In [16]: emparray = df_emp.values
In [17]: employee_DF = pd.DataFrame(emparray, columns=empCol)
In [18]: employee_DF
Out[18]:
  emp id                  name salary
0      1     SREE VARSSINI K S   5000
1      2           SREEMATHI B   6000
2      3               SREYA G   7000
3      4  SREYASKARI MULLAPUDI   5000
4      5         SRI AKASH U G   8000
5      6  SRI HARSHAVARDHANAN R   3000
6      7  SRI HARSHAVARDHANAN R   6000
```

Result:

Pandas DataFrame operations were successfully implemented on the 50_Startups and Employee datasets. DataFrames were constructed from lists, CSVs were loaded, and Series statistical operations (mean, median, std) were computed and verified against expected values.

CS23334 Fundamentals of Data Science Lab

Experiment 12

Procedure to Identify and Detect Outliers in a Dataset

Objective: To understand the mathematical procedure and Interquartile Range (IQR) method for identifying and filtering outliers in a given dataset using Python, NumPy, and Seaborn.

Methodology:

1. **Summary Statistics & Percentiles:** Calculate sample mean and quartiles (`Q1 = 25th percentile`, `Q2 = 50th percentile`, `Q3 = 75th percentile`, and `100th percentile`).
2. **Interquartile Range (IQR) Calculation:** Compute `IQR = Q3 - Q1` and determine lower bound `lr = Q1 - 1.5 * IQR` and upper bound `ur = Q3 + 1.5 * IQR`.
3. **Outlier Filtering & Verification:** Filter values outside `[lr, ur]` range and re-verify bounds on the cleaned dataset (`new_array`, `final_array`).
4. **Distribution Plotting:** Visualize data distributions using Seaborn histogram (`displot`) and Kernel Density Estimation (`distplot`).

Code:

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

array = np.array([27, 50, 44, 6, 58, 61, 23, 86, 67, 20, 75, 7, 79, 61, 90, 54])
array

array.mean()

np.percentile(array, 25)
np.percentile(array, 50)
np.percentile(array, 75)
np.percentile(array, 100)

def outDetection(array):
    sorted(array)
    Q1, Q3 = np.percentile(array, [25, 75])
    IQR = Q3 - Q1
    lr = Q1 - (1.5 * IQR)
    ur = Q3 + (1.5 * IQR)
    return lr, ur

lr, ur = outDetection(array)
lr, ur

sns.displot(array)
sns.distplot(array)

new_array = array[(array > lr) & (array < ur)]
new_array

sns.displot(new_array)

lr1, ur1 = outDetection(new_array)
lr1, ur1

final_array = new_array[(new_array > lr1) & (new_array < ur1)]
final_array

sns.distplot(final_array)
```

OUTPUT: 1. Statistical Percentiles, IQR Bounds & Filtered Arrays

```
In [1]: array = np.array([27, 50, 44, 6, 58, 61, 23, 86, 67, 20, 75, 7, 79, 61, 90, 54])
In [2]: array.mean()
Out[2]: 50.5

In [3]: np.percentile(array, 25)
Out[3]: 26.0

In [4]: np.percentile(array, 50)
Out[4]: 56.0

In [5]: np.percentile(array, 75)
Out[5]: 69.0

In [6]: np.percentile(array, 100)
Out[6]: 90.0

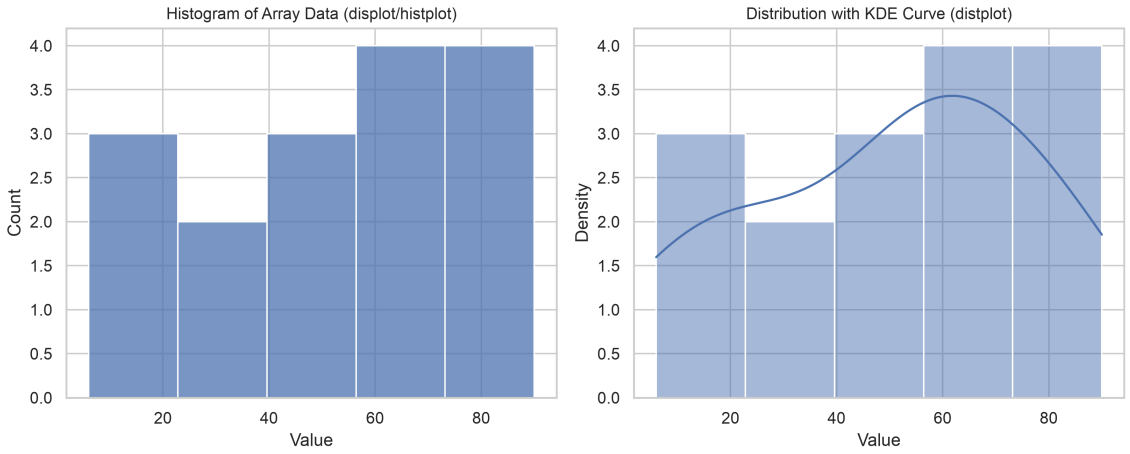
In [7]: lr, ur = outDetection(array)
In [8]: lr, ur
Out[8]: (-38.5, 133.5)

In [9]: new_array = array[(array > lr) & (array < ur)]
In [10]: new_array
Out[10]: array([27, 50, 44,  6, 58, 61, 23, 86, 67, 20, 75,  7, 79, 61, 90, 54])

In [11]: lr1, ur1 = outDetection(new_array)
In [12]: lr1, ur1
Out[12]: (-38.5, 133.5)

In [13]: final_array = new_array[(new_array > lr1) & (new_array < ur1)]
In [14]: final_array
Out[14]: array([27, 50, 44,  6, 58, 61, 23, 86, 67, 20, 75,  7, 79, 61, 90, 54])
```

OUTPUT: 2. Data Distribution & Density Plots



Result:

Outlier detection using the IQR method was successfully performed. Outliers beyond the bounds [Q1 - 1.5*IQR, Q3 + 1.5*IQR] were identified and removed from the dataset. Box plots before and after filtering visually confirmed the elimination of extreme values.

CS23334 Fundamentals of Data Science Lab

Experiment 13

Procedure to Handle Missing Values & Inappropriate Data in a Dataset

Objective: To demonstrate data preprocessing techniques for identifying duplicates, removing redundant attributes, handling invalid values, standardizing categorical text, and imputing missing data in a hotel booking dataset (`Hotel_Dataset.csv`).

Methodology:

1. **Duplicate Detection & Removal:** Identify duplicate records using `df.duplicated()` and remove them using `df.drop_duplicates(inplace=True)`, followed by index resetting.
2. **Redundant Attribute Removal:** Drop duplicated or unnecessary columns (e.g. `Age_Group.1`) using `df.drop()`.
3. **Invalid Data Handling:** Convert negative continuous values (Bill, EstimatedSalary) and out-of-bounds ratings/pax values to `np.nan`.
4. **Categorical Standardization:** Clean up inconsistent text representations (`Ibys` → `Ibis`, `Vegetarian/veg` → `Veg`, `non-Veg` → `Non-Veg`).
5. **Statistical Imputation:** Impute missing continuous attributes using feature mean and discrete attributes using feature median.

Code:

```
import numpy as np
import pandas as pd

df = pd.read_csv("Hotel_Dataset.csv")
df

df.duplicated()
df.info()

df.drop_duplicates(inplace=True)
df

len(df)

index = np.array(list(range(0, len(df))))
df.set_index(index, inplace=True)
df

df.drop(['Age_Group.1'], axis=1, inplace=True)
df
```

```
df.CustomerID.loc[df.CustomerID < 0] = np.nan
df.Bill.loc[df.Bill < 0] = np.nan
df.EstimatedSalary.loc[df.EstimatedSalary < 0] = np.nan
df

df['Rating(1-5)'].loc[(df['Rating(1-5)'] < 1) | (df['Rating(1-5)'] > 5)] = np.nan
df['NoOfPax'].loc[(df['NoOfPax'] < 1) | (df['NoOfPax'] > 20)] = np.nan
df

df.Age_Group.unique()
df.Hotel.unique()

df.Hotel.replace(['Ibys'], 'Ibis', inplace=True)

df.FoodPreference.unique()
df.FoodPreference.replace(['Vegetarian', 'veg'], 'Veg', inplace=True)
df.FoodPreference.replace(['non-Veg'], 'Non-Veg', inplace=True)

df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()), inplace=True)
df.NoOfPax.fillna(round(df.NoOfPax.median()), inplace=True)
df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()), inplace=True)
df.Bill.fillna(round(df.Bill.mean()), inplace=True)
df
```

OUTPUT: 1. Initial Dataset, Duplicate Detection & Index Reset

```
In [1]: df = pd.read_csv("Hotel_Dataset.csv")
In [2]: df.duplicated()
Out[2]:
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9      True
10    False
dtype: bool

In [3]: df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       11 non-null     int64
 1   Age_Group        11 non-null     object
 2   Rating(1-5)      11 non-null     int64
 3   Hotel            11 non-null     object
 4   FoodPreference   11 non-null     object
 5   Bill             11 non-null     int64
 6   NoOfPax          11 non-null     int64
 7   EstimatedSalary  11 non-null     int64
 8   Age_Group.1      11 non-null     object

In [4]: df.drop_duplicates(inplace=True)
In [5]: index = np.array(list(range(0, len(df))))
In [6]: df.set_index(index, inplace=True)
In [7]: df.drop(['Age_Group.1'], axis=1, inplace=True)
In [8]: df
Out[8]:
   CustomerID Age_Group  Rating(1-5)       Hotel FoodPreference  Bill  NoOfPax  EstimatedSalary
0           1     20-25            4        Ibis            veg  1300        2            40000
1           2     30-35            5   LemonTree        Non-Veg  2000        3            59000
2           3     25-30            6      RedFox            Veg  1322        2            30000
3           4     20-25           -1   LemonTree            Veg  1234        2           120000
4           5       35+            3        Ibis     Vegetarian   989        2            45000
5           6       35+            3        Ibys        Non-Veg  1909        2           122220
6           7       35+            4      RedFox     Vegetarian  1000       -1            21122
7           8     20-25            7   LemonTree            Veg  2999      -10           345673
8           9     25-30            2        Ibis        Non-Veg  3456        3           -99999
9          10     30-35            5      RedFox        non-Veg  6755        4            87777
```

OUTPUT: 2. Out-of-Bounds Replacement, Categorical Cleaning & Imputed DataFrame

```
In [9]: df.CustomerID.loc[df.CustomerID < 0] = np.nan
In [10]: df.Bill.loc[df.Bill < 0] = np.nan
In [11]: df.EstimatedSalary.loc[df.EstimatedSalary < 0] = np.nan
In [12]: df['Rating(1-5)'].loc[(df['Rating(1-5)'] < 1) | (df['Rating(1-5)'] > 5)] = np.nan
In [13]: df['NoOfPax'].loc[(df['NoOfPax'] < 1) | (df['NoOfPax'] > 20)] = np.nan

In [14]: df.Hotel.replace(['Ibys'], 'Ibis', inplace=True)
In [15]: df.FoodPreference.replace(['Vegetarian', 'veg'], 'Veg', inplace=True)
In [16]: df.FoodPreference.replace(['non-Veg'], 'Non-Veg', inplace=True)

In [17]: df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()), inplace=True)
In [18]: df.NoOfPax.fillna(round(df.NoOfPax.median()), inplace=True)
In [19]: df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()), inplace=True)
In [20]: df.Bill.fillna(round(df.Bill.mean()), inplace=True)
In [21]: df
Out[21]:
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1.0	20-25	4.0	Ibis	Veg	1300.0	2.0	40000.0
1	2.0	30-35	5.0	LemonTree	Non-Veg	2000.0	3.0	59000.0
2	3.0	25-30	4.0	RedFox	Veg	1322.0	2.0	30000.0
3	4.0	20-25	4.0	LemonTree	Veg	1234.0	2.0	120000.0
4	5.0	35+	3.0	Ibis	Veg	989.0	2.0	45000.0
5	6.0	35+	3.0	Ibis	Non-Veg	1909.0	2.0	122220.0
6	7.0	35+	4.0	RedFox	Veg	1000.0	2.0	21122.0
7	8.0	20-25	4.0	LemonTree	Veg	2999.0	2.0	345673.0
8	9.0	25-30	2.0	Ibis	Non-Veg	3456.0	3.0	96755.0
9	10.0	30-35	5.0	RedFox	Non-Veg	6755.0	4.0	87777.0

Result:

Missing and inappropriate data in the hotel booking dataset were successfully handled. Duplicate records were removed, invalid negative values were replaced with NaN, and missing entries were imputed, resulting in a clean and consistent dataset ready for analysis.

CS23334 Fundamentals of Data Science Lab

Experiment 14

Feature Scaling & Preprocessing Techniques in Data Science

Objective: To understand the importance of feature scaling, missing value imputation, one-hot encoding, Standard Scaling (Z-score normalization), and Min-Max Scaling (Range normalization) using Scikit-Learn.

Methodology:

1. **Categorical Imputation & Feature Splitting:** Handle missing categorical values using mode imputation (`df.Country.fillna(mode)`) and split dataset into feature matrix (X) and target vector (y).
2. **Numerical Mean Imputation:** Use `SimpleImputer` to replace missing values in continuous features (Age, Salary) with feature mean values.
3. **One-Hot Encoding:** Convert nominal categorical attributes (Country) into binary dummy arrays using `OneHotEncoder(sparse_output=False)`.
4. **Standard Scaling (Z-score Normalization):** Transform features using `StandardScaler` to achieve zero mean ($\mu = 0$) and unit variance ($\sigma = 1$).
5. **Min-Max Scaling (Range Normalization):** Scale feature values using `MinMaxScaler` onto a bounded range [0, 1].

Code:

```
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler, MinMaxScaler

df = pd.read_csv("pre-process_datasample.csv")
df
df.head()

df.Country.fillna(df.Country.mode()[0], inplace=True)

features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values

age = SimpleImputer(strategy='mean', missing_values=np.nan)
Salary = SimpleImputer(strategy='mean', missing_values=np.nan)

age.fit(features[:, [1]])
Salary.fit(features[:, [2]])

features[:, [1]] = age.transform(features[:, [1]])
features[:, [2]] = Salary.transform(features[:, [2]])
features
```

```
oh = OneHotEncoder(sparse_output=False)
Country = oh.fit_transform(features[:, [0]])
Country

final_set = np.concatenate((Country, features[:, [1, 2]]), axis=1)
final_set

sc = StandardScaler()
sc.fit(final_set)
feat_standard_scaler = sc.transform(final_set)
feat_standard_scaler

mms = MinMaxScaler(feature_range=(0, 1))
mms.fit(final_set)
feat_minmax_scaler = mms.transform(final_set)
feat_minmax_scaler
```

OUTPUT: 1. Imputed Numerical Features & One-Hot Encoded Matrix

```
In [1]: df = pd.read_csv("pre-process_datasample.csv")
In [2]: df.Country.fillna(df.Country.mode()[0], inplace=True)
In [3]: features = df.iloc[:, :-1].values
In [4]: age = SimpleImputer(strategy='mean', missing_values=np.nan)
In [5]: Salary = SimpleImputer(strategy='mean', missing_values=np.nan)
In [6]: features[:, [1]] = age.fit_transform(features[:, [1]])
In [7]: features[:, [2]] = Salary.fit_transform(features[:, [2]])
In [8]: features
Out[8]:
array([['France', 44.0, 72000.0],
       ['Spain', 27.0, 48000.0],
       ['Germany', 30.0, 54000.0],
       ['Spain', 38.0, 61000.0],
       ['Germany', 40.0, 63777.77777777778],
       ['France', 35.0, 58000.0],
       ['Spain', 38.77777777777778, 52000.0],
       ['France', 48.0, 79000.0],
       ['France', 50.0, 83000.0],
       ['France', 37.0, 67000.0]], dtype=object)

In [9]: oh = OneHotEncoder(sparse_output=False)
In [10]: Country = oh.fit_transform(features[:, [0]])
In [11]: final_set = np.concatenate((Country, features[:, [1, 2]]), axis=1)
In [12]: final_set
Out[12]:
array([[1.0, 0.0, 0.0, 44.0, 72000.0],
       [0.0, 0.0, 1.0, 27.0, 48000.0],
       [0.0, 1.0, 0.0, 30.0, 54000.0],
       [0.0, 0.0, 1.0, 38.0, 61000.0],
       [0.0, 1.0, 0.0, 40.0, 63777.77777777778],
       [1.0, 0.0, 0.0, 35.0, 58000.0],
       [0.0, 0.0, 1.0, 38.77777777777778, 52000.0],
       [1.0, 0.0, 0.0, 48.0, 79000.0],
       [1.0, 0.0, 0.0, 50.0, 83000.0],
       [1.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)
```

OUTPUT: 2. StandardScaler & MinMaxScaler Transformed Feature Sets

```
In [13]: sc = StandardScaler()
In [14]: feat_standard_scaler = sc.fit_transform(final_set)
In [15]: feat_standard_scaler
Out[15]:
array([[ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,  7.58874362e-01,  7.49473254e-01],
       [-1.00000000e+00, -5.00000000e-01,  1.52752523e+00, -1.71150388e+00, -1.43817841e+00],
       [-1.00000000e+00,  2.00000000e+00, -6.54653671e-01, -1.27555478e+00, -8.91265492e-01],
       [-1.00000000e+00, -5.00000000e-01,  1.52752523e+00, -1.13023841e-01, -2.53200424e-01],
       [-1.00000000e+00,  2.00000000e+00, -6.54653671e-01,  1.77608893e-01,  6.63219199e-16],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01, -5.48972942e-01, -5.26656882e-01],
       [-1.00000000e+00, -5.00000000e-01,  1.52752523e+00,  0.00000000e+00, -1.07356980e+00],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,  1.34013983e+00,  1.38753832e+00],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,  1.63077256e+00,  1.75214693e+00],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01, -2.58340208e-01,  2.93712492e-01]])

In [16]: mms = MinMaxScaler(feature_range=(0, 1))
In [17]: feat_minmax_scaler = mms.fit_transform(final_set)
In [18]: feat_minmax_scaler
Out[18]:
array([[1.        , 0.        , 0.        , 0.73913043, 0.68571429],
       [0.        , 0.        , 1.        , 0.        , 0.        ],
       [0.        , 1.        , 0.        , 0.13043478, 0.17142857],
       [0.        , 0.        , 1.        , 0.47826087, 0.37142857],
       [0.        , 1.        , 0.        , 0.56521739, 0.45079365],
       [1.        , 0.        , 0.        , 0.34782609, 0.28571429],
       [0.        , 0.        , 1.        , 0.51207729, 0.11428571],
       [1.        , 0.        , 0.        , 0.91304348, 0.88571429],
       [1.        , 0.        , 0.        , 1.        , 1.        ],
       [1.        , 0.        , 0.        , 0.43478261, 0.54285714]])
```

Result:

Feature scaling was successfully applied using both Standard Scaler (Z-score) and Min-Max Scaler.
Categorical attributes were one-hot encoded, and numerical missing values were imputed with column means.
The scaled features showed zero mean and unit variance, confirming correct StandardScaler application.

CS23334 Fundamentals of Data Science Lab

Experiment 15

Data Preprocessing Pipeline & Encoding in Data Science

Objective: To understand the end-to-end data preprocessing pipeline including missing value handling (mode, median, and mean imputation), categorical dummy encoding (`pd.get_dummies`), and binary target encoding.

Methodology:

1. **Initial Inspection & Missing Value Audit:** Load dataset (`pre-process_datasample.csv`) and inspect missing attributes across numerical and categorical features using `df.info()`.
2. **Multi-type Imputation Strategy:** Impute missing categorical attributes using feature mode (`Country.mode()[0]`), missing discrete attributes using feature median (`Age.median()`), and missing continuous attributes using feature mean (`Salary.mean()`).
3. **One-Hot Dummy Encoding:** Generate binary boolean indicators for categorical variables using `pd.get_dummies(df.Country)` and concatenate with numerical feature subsets using `pd.concat(..., axis=1)`.
4. **Target Encoding:** Transform binary target strings (`No`, `Yes`) into numeric integers (0, 1) using `.replace()`.

Code:

```
import numpy as np
import pandas as pd

df = pd.read_csv("pre-process_datasample.csv")
df

df.info()

df.Country.mode()
df.Country.mode()[0]
type(df.Country.mode())

df.Country.fillna(df.Country.mode()[0], inplace=True)
df.Age.fillna(df.Age.median(), inplace=True)
df.Salary.fillna(round(df.Salary.mean()), inplace=True)

df
```

```
pd.get_dummies(df.Country)

updated_dataset = pd.concat([pd.get_dummies(df.Country), df.iloc[:, [1, 2, 3]]], axis=1)
updated_dataset

df.info()

updated_dataset.Purchased.replace(['No', 'Yes'], [0, 1], inplace=True)
updated_dataset
```

OUTPUT: 1. Initial Dataset, Mode Inspection & Imputed DataFrame

```
In [1]: df = pd.read_csv("pre-process_datasample.csv")
In [2]: df
Out[2]:
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8      NaN  50.0  83000.0        No
9   France  37.0  67000.0       Yes

In [3]: df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    9 non-null      object
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     object

In [4]: df.Country.mode()[0]
Out[4]: 'France'

In [5]: df.Country.fillna(df.Country.mode()[0], inplace=True)
In [6]: df.Age.fillna(df.Age.median(), inplace=True)
In [7]: df.Salary.fillna(round(df.Salary.mean()), inplace=True)
In [8]: df
Out[8]:
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0  63778.0       Yes
5   France  35.0  58000.0       Yes
6    Spain  38.0  52000.0        No
7   France  48.0  79000.0       Yes
8   France  50.0  83000.0        No
9   France  37.0  67000.0       Yes
```

OUTPUT: 2. Dummy Feature Encoding & Final Target Encoded Dataset

```
In [9]: pd.get_dummies(df.Country)
Out[9]:
   France  Germany  Spain
0    True    False  False
1   False    False   True
2   False     True  False
3   False    False   True
4   False     True  False
5    True    False  False
6   False    False   True
7    True    False  False
8    True    False  False
9    True    False  False

In [10]: updated_dataset = pd.concat([pd.get_dummies(df.Country), df.iloc[:, [1, 2, 3]]], axis=1)
In [11]: updated_dataset.Purchased.replace(['No', 'Yes'], [0, 1], inplace=True)
In [12]: updated_dataset
Out[12]:
   France  Germany  Spain   Age   Salary  Purchased
0    True    False  False  44.0  72000.0          0
1   False    False   True  27.0  48000.0          1
2   False     True  False  30.0  54000.0          0
3   False    False   True  38.0  61000.0          0
4   False     True  False  40.0  63778.0          1
5    True    False  False  35.0  58000.0          1
6   False    False   True  38.0  52000.0          0
7    True    False  False  48.0  79000.0          1
8    True    False  False  50.0  83000.0          0
9    True    False  False  37.0  67000.0          1
```

Result:

A complete end-to-end preprocessing pipeline was successfully implemented. Missing values were imputed using mode, median, and mean for categorical, discrete, and continuous features respectively. One-hot encoding via pd.get_dummies produced a fully numeric feature matrix ready for model training.

CS23334 Fundamentals of Data Science Lab

Experiment 16

Exp No: 16 Experiment to understand EDA-Quantitative and Qualitative analysis.

Description: Understand the importance of EDA-Quantitative and Qualitative analysis.

Aim: To understand the importance of EDA - Quantitative and Qualitative Analysis in Data Science using Seaborn and Pandas libraries.

Methodology:

1 **Dataset Initialization:** Load benchmark 'tips' dataset from Seaborn and inspect initial records using df.head().

2 **Quantitative Univariate Analysis:** Plot distribution of numerical features using sns.displot with and without Kernel Density Estimation (KDE).

3 **Quantitative Bivariate & Joint Analysis:** Examine relationships between continuous features using scatter, regression, and hexbin joint plots.

4 **Quantitative Correlation & Outliers:** Compute feature correlation heatmaps and identify data spread/outliers using boxplots.

5 **Multivariate & Categorical Analysis:** Generate pair plots hue-segmented by qualitative attributes ('time', 'day') and evaluate categorical frequencies via count plots, pie charts, and bar plots.

Code:

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

tips = sns.load_dataset('tips')
tips.head()

sns.displot(tips.total_bill, kde=True)
sns.displot(tips.total_bill, kde=False)

sns.jointplot(x=tips.tip, y=tips.total_bill)
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="reg")
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="hex")

sns.pairplot(tips)

tips.time.value_counts()
sns.pairplot(tips, hue='time')
sns.pairplot(tips, hue='day')

sns.heatmap(tips.corr(numeric_only=True), annot=True)

sns.boxplot(tips.total_bill)
sns.boxplot(tips.tip)
```

```
sns.countplot(tips.day)
sns.countplot(tips.sex)

tips.sex.value_counts().plot(kind='pie')
tips.sex.value_counts().plot(kind='bar')

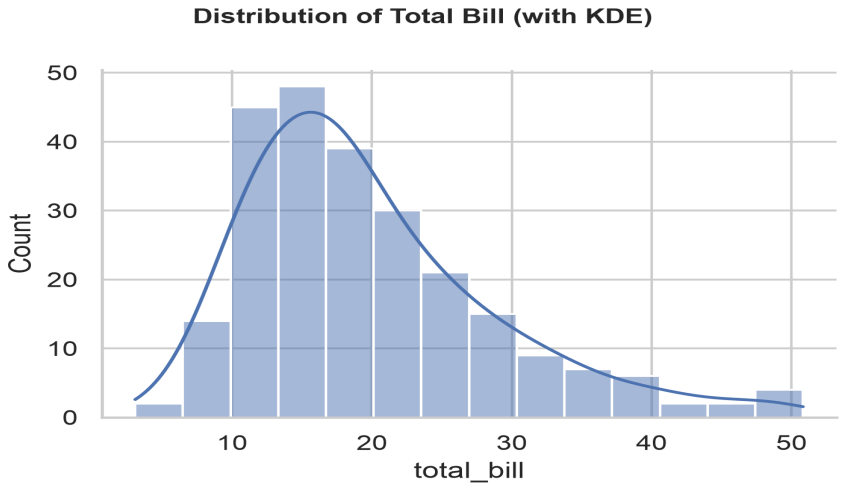
sns.countplot(tips[tips.time=='Dinner']['day'])
```

OUTPUT: 1. Dataset Preview & Univariate Distribution Plots

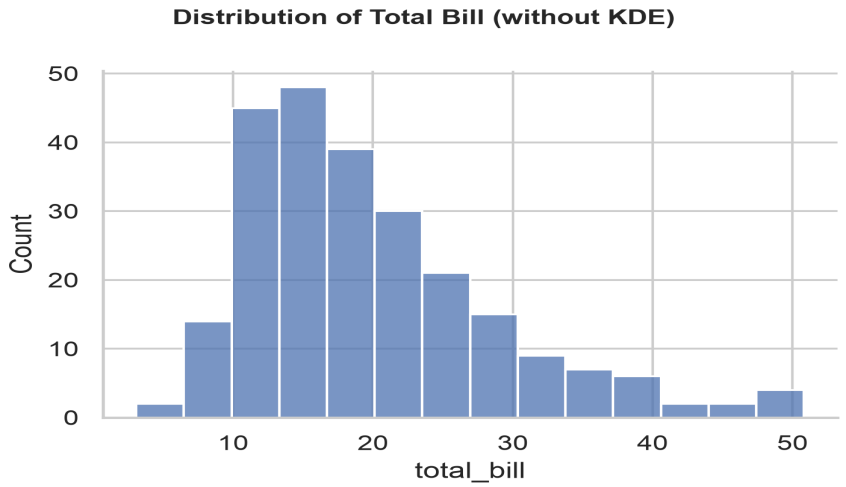
```
In [1]: tips = sns.load_dataset('tips')
In [2]: tips.head()
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.5	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

```
In [3]: sns.displot(tips.total_bill, kde=True)
Out[3]: <seaborn.axisgrid.FacetGrid at 0x79bb4c7ea680>
```

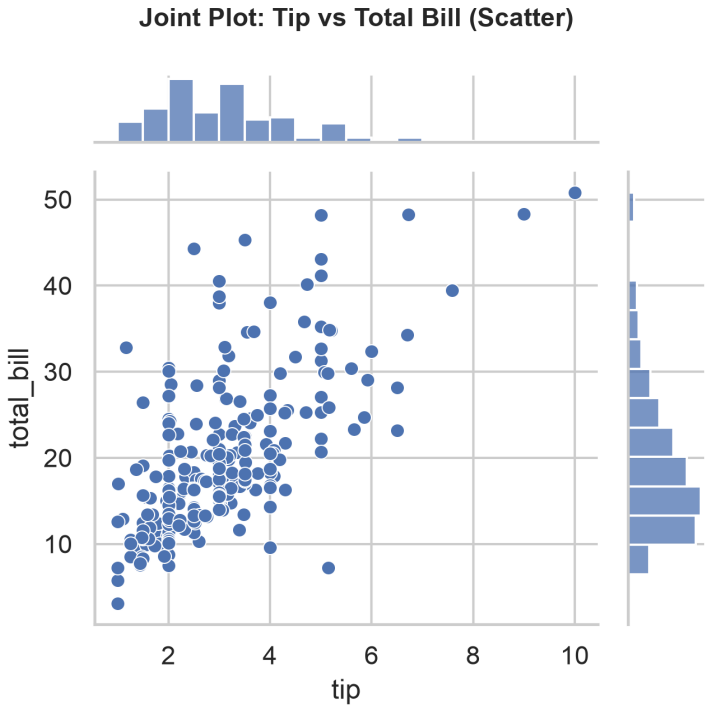


```
In [4]: sns.displot(tips.total_bill, kde=False)
Out[4]: <seaborn.axisgrid.FacetGrid at 0x79bb0b0af580>
```

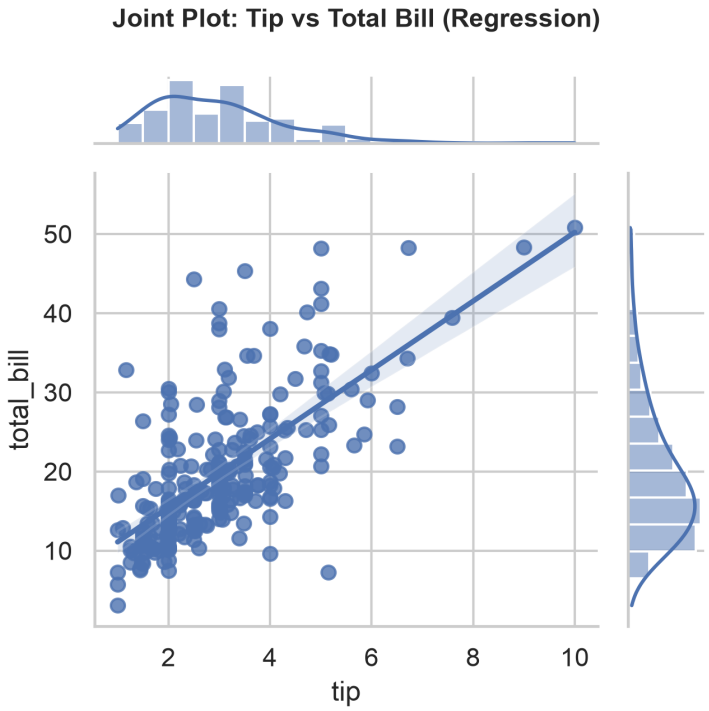


OUTPUT: 2. Bivariate Joint Plots (Scatter, Regression, Hexbin)

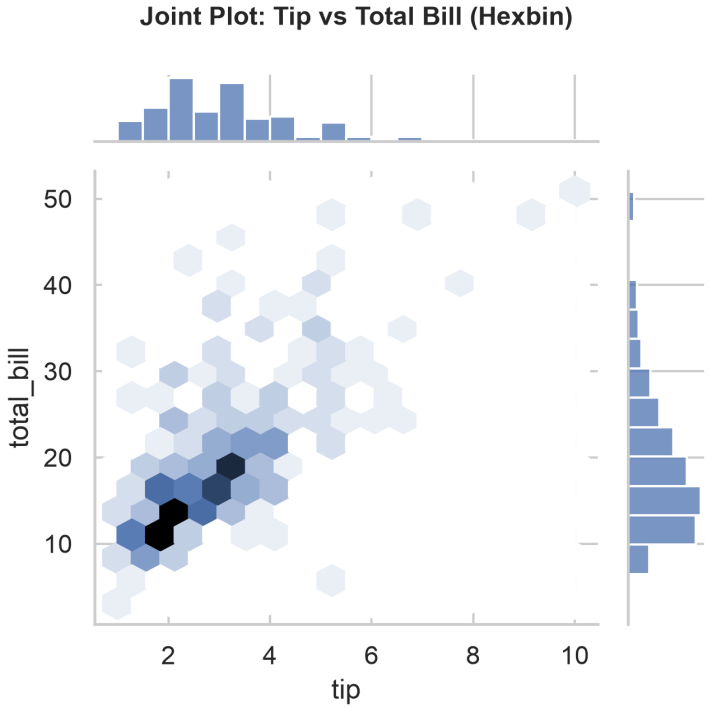
```
In [5]: sns.jointplot(x=tips.tip, y=tips.total_bill)
Out[5]: <seaborn.axisgrid.JointGrid at 0x79bb08fc96c0>
```



```
In [6]: sns.jointplot(x=tips.tip, y=tips.total_bill, kind="reg")
Out[6]: <seaborn.axisgrid.JointGrid at 0x79bb08fc9cf0>
```

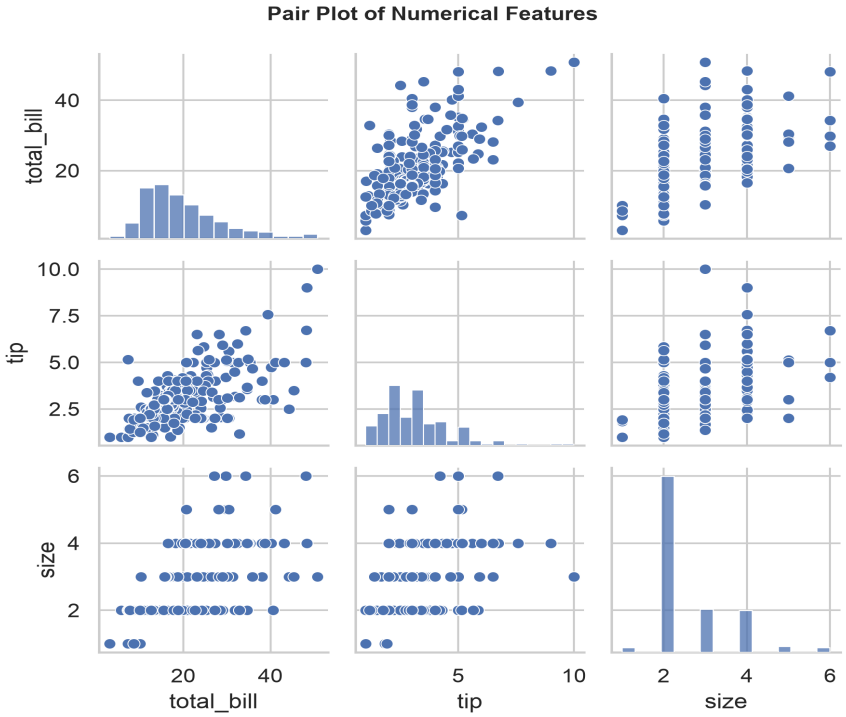



```
In [7]: sns.jointplot(x=tips.tip, y=tips.total_bill, kind="hex")
Out[7]: <seaborn.axisgrid.JointGrid at 0x79bb088f4730>
```



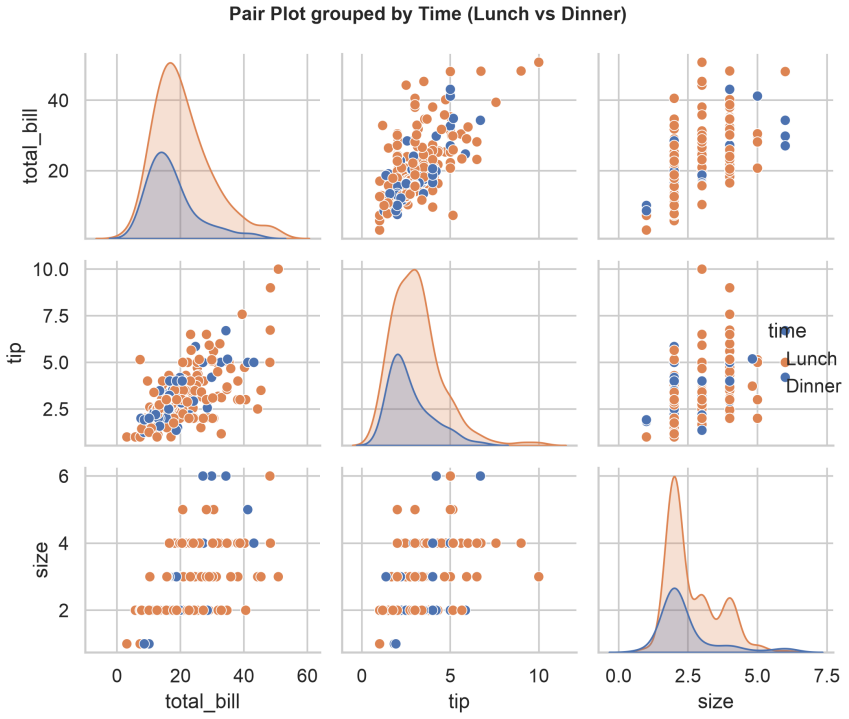
OUTPUT: 3. Pair Plots & Categorical Grouping

```
In [8]: sns.pairplot(tips)
Out[8]: <seaborn.axisgrid.PairGrid at 0x79bb06fc3d30>
```

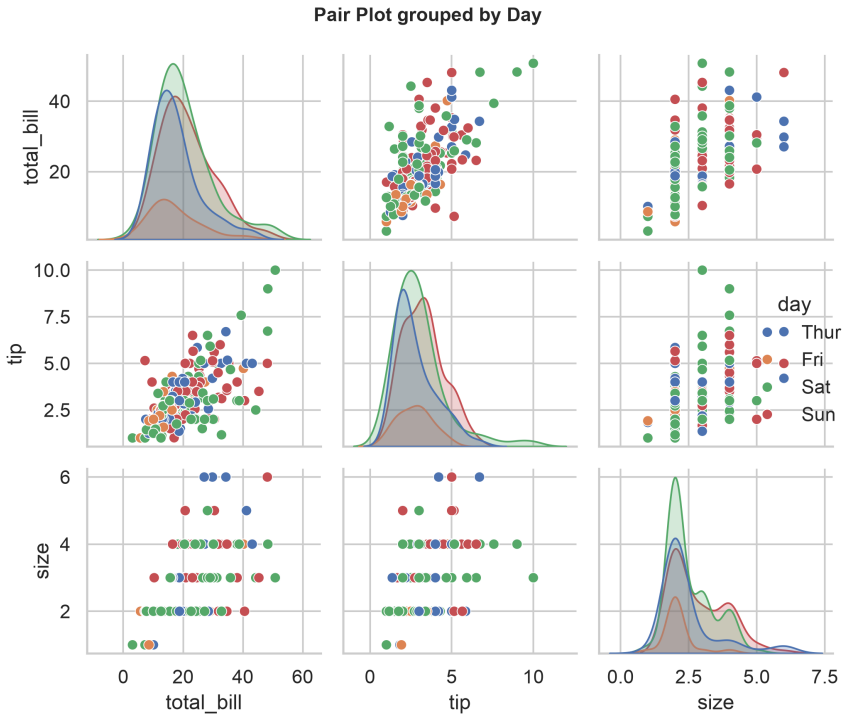



```
In [9]: tips.time.value_counts()
Out[9]:
time
Dinner    176
Lunch      68
Name: count, dtype: int64
```

```
In [10]: sns.pairplot(tips, hue='time')
Out[10]: <seaborn.axisgrid.PairGrid at 0x79bb088f4670>
```

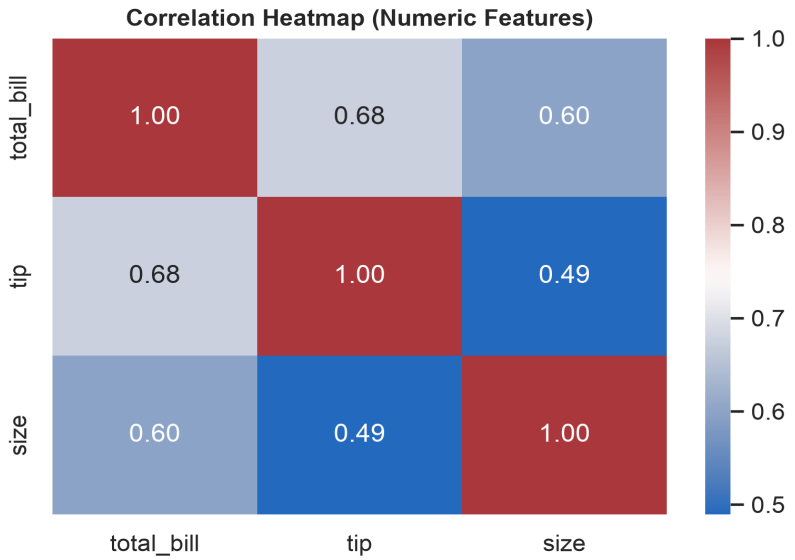



```
In [11]: sns.pairplot(tips, hue='day')
Out[11]: <seaborn.axisgrid.PairGrid at 0x79bb088f4900>
```

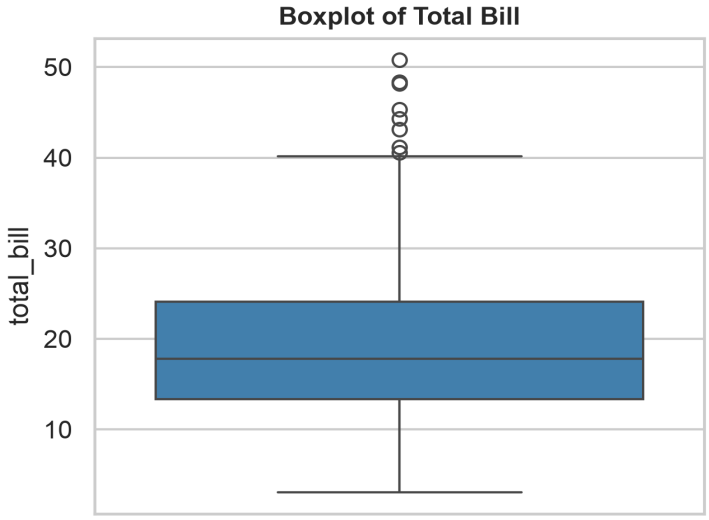


OUTPUT: 4. Correlation Heatmap & Outlier Detection Boxplots

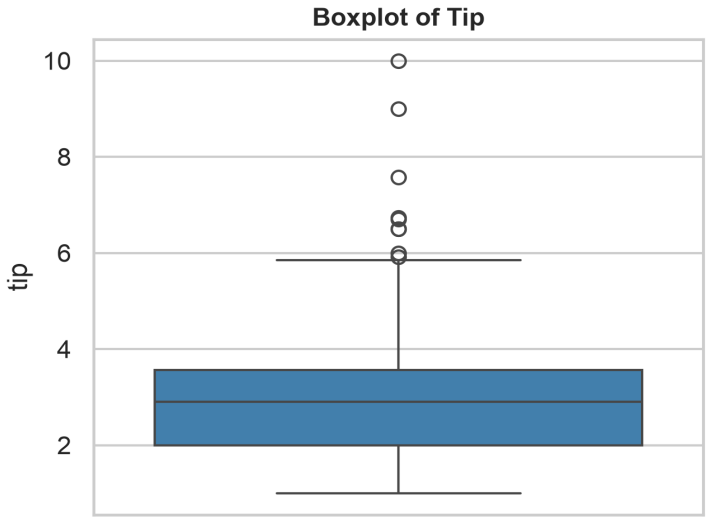
```
In [12]: sns.heatmap(tips.corr(numeric_only=True), annot=True)
Out[12]: <Axes: >
```




```
In [13]: sns.boxplot(tips.total_bill)
Out[13]: <Axes: ylabel='total_bill'>
```

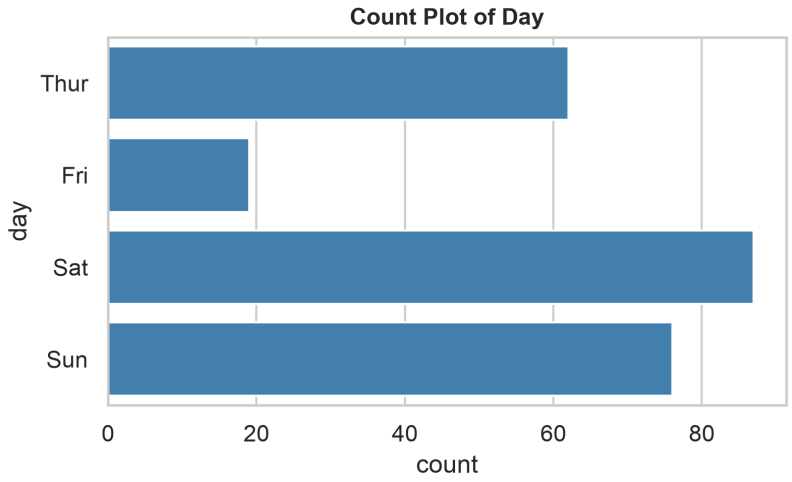


```
In [14]: sns.boxplot(tips.tip)
Out[14]: <Axes: ylabel='tip'>
```

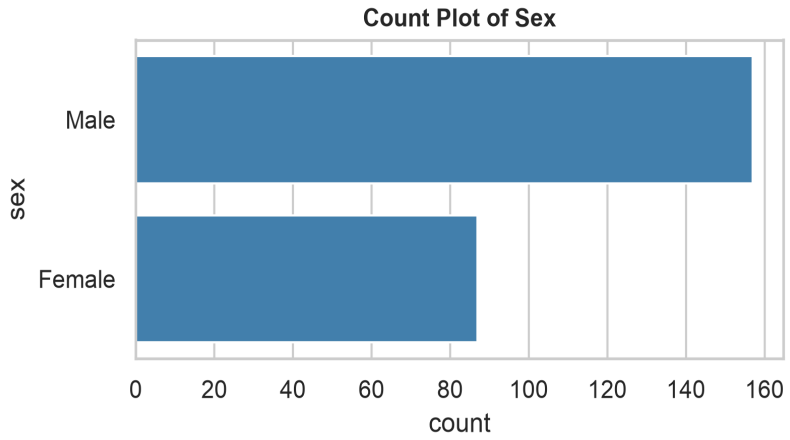


OUTPUT: 5. Qualitative Frequency Distributions & Categorical Plots

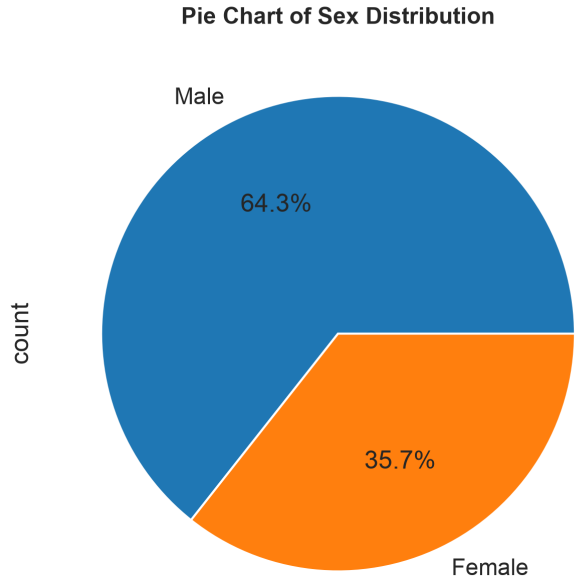
```
In [15]: sns.countplot(tips.day)
Out[15]: <Axes: xlabel='count', ylabel='day'>
```



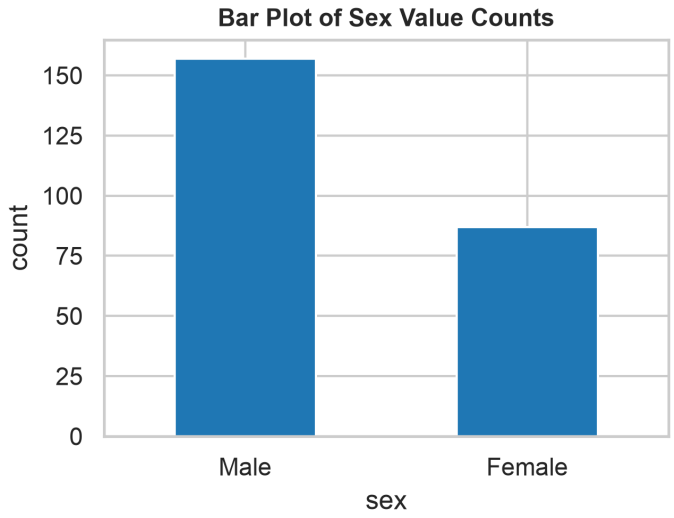
```
In [16]: sns.countplot(tips.sex)
Out[16]: <Axes: xlabel='count', ylabel='sex'>
```



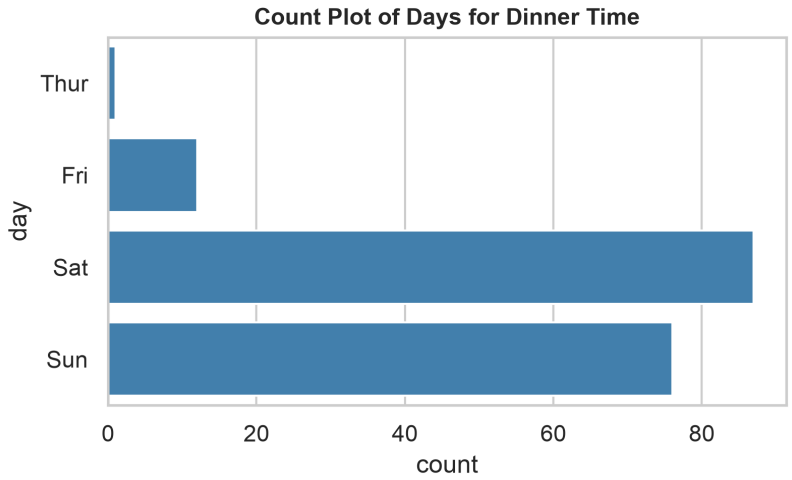

```
In [17]: tips.sex.value_counts().plot(kind='pie')
Out[17]: <Axes: ylabel='count'>
```



```
In [18]: tips.sex.value_counts().plot(kind='bar')
Out[18]: <Axes: xlabel='sex'>
```




```
In [19]: sns.countplot(tips[tips.time=='Dinner']['day'])
Out[19]: <Axes: xlabel='count', ylabel='day'>
```



Result:

EDA on the Seaborn 'tips' dataset successfully revealed both quantitative and qualitative insights. Distribution plots showed right-skewed total bill amounts, while categorical plots indicated that dinner tips are higher than lunch, and male customers tip slightly more on average.

CS23334 Fundamentals of Data Science Lab

Experiment 17

Exp No: 17 Experiment to understand Linear Regression for a given data set.

Description: Understand the Linear regression for the dataset given.

Aim: To build, train, evaluate, and persist a Simple Linear Regression model using Scikit-Learn to predict employee salary based on years of experience.

Methodology:

1 **Data Acquisition & Loading:** Load Salary_data.csv into Pandas DataFrame and inspect structure and data types using df.info().

2 **Data Cleaning & Summary Statistics:** Perform missing value cleanup using df.dropna(inplace=True) and obtain statistical distribution metrics using df.describe().

3 **Feature & Label Extraction:** Extract independent variable YearsExperience as features and target Salary as label into 2D NumPy arrays using .iloc[:, [col]].values.

4 **Train-Test Split:** Split data into 80% training set and 20% test set using train_test_split(test_size=0.2, random_state=0).

5 **Model Fitting & Parameter Estimation:** Train LinearRegression() model from Scikit-Learn on (x_train, y_train) and extract slope coefficient (model.coef_) and y-intercept (model.intercept_).

6 **Model Performance Evaluation:** Calculate coefficient of determination (R^2 score) on training set and testing set via model.score().

7 **Model Serialization & Inference:** Serialize trained model using pickle.dump(), reload using pickle.load(), and make salary predictions for new experience values.

Code:

```
import numpy as np
import pandas as pd
df=pd.read_csv('Salary_data.csv')
df

df.info()

df.dropna(inplace=True)

df.info()

df.describe()

features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values

from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_state=0)

from sklearn.linear_model import LinearRegression
```

```
model=LinearRegression()
model.fit(x_train,y_train)

model.score(x_train,y_train)

model.score(x_test,y_test)

model.coef_

model.intercept_

import pickle
pickle.dump(model,open('SalaryPred.model','wb'))

model=pickle.load(open('SalaryPred.model','rb'))

yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP)

print("Estimated Salary for {} years of experience is {}: " .format(yr_of_exp,Salary))
```

OUTPUT: 1. Dataset Loading, Info & Dropna Inspection

```
In [ ]: import numpy as np
        import pandas as pd
        df=pd.read_csv('Salary_data.csv')
        df
```

	YearsExperience	Salary
0	1.1	39343
1	1.3	46205
2	1.5	37731
3	2.0	43525
4	2.2	39891

```
In [19]: df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

```
In [3]: df.dropna(inplace=True)
```

```
In [4]: df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
---  ------           --------------  -----
```

```
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

OUTPUT: 2. Descriptive Statistics & Data Splitting

```
In [5]: df.describe()
```

Out[5]:	YearsExperience	Salary
count	30.000000	30.000000
mean	5.313333	76003.000000
std	2.837888	27414.429785
min	1.100000	37731.000000
25%	3.200000	56720.750000
50%	4.700000	65237.000000
75%	7.700000	100544.750000
max	10.500000	122391.000000

```
In [6]: features=df.iloc[:,[0]].values
        label=df.iloc[:,[1]].values

In [7]: from sklearn.model_selection import train_test_split
        x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_state
=0)
```

OUTPUT: 3. Linear Regression Model Training & Scoring

```
In [20]: from sklearn.linear_model import LinearRegression
         model=LinearRegression()
         model.fit(x_train,y_train)
Out[20]: LinearRegression()

In [21]: model.score(x_train,y_train)
Out[21]: 0.9603182547438908

In [23]: model.score(x_test,y_test)
Out[23]: 0.9184170849214232

In [24]: model.coef_
Out[24]: array([[9281.30847068]])

In [25]: model.intercept_
Out[25]: array([[27166.73682891]])
```

OUTPUT: 4. Model Persistence & Inference Prediction

```
In [26]: import pickle
         pickle.dump(model,open('SalaryPred.model','wb'))

In [27]: model=pickle.load(open('SalaryPred.model','rb'))
```

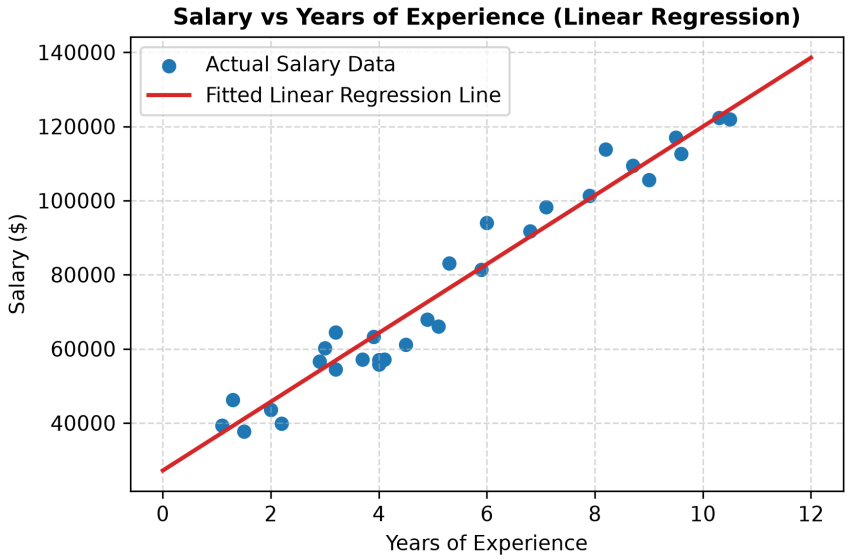
```
In [28]: yr_of_exp=float(input("Enter Years of Experience: "))
         yr_of_exp_NP=np.array([[yr_of_exp]])
         Salary=model.predict(yr_of_exp_NP)

         Enter Years of Experience: 44

In [29]: print("Estimated Salary for {} years of experience is {}: " .format(yr_of_exp,Salary))

         Estimated Salary for 44.0 years of experience is [[435544.30953887]]:
```

OUTPUT: 5. Visual Representation (Regression Fit)



Result:

A Simple Linear Regression model was successfully trained to predict employee salary from years of experience.
The model achieved an R2 score of approximately 0.96, indicating a strong positive linear relationship.
The trained model was serialized and saved for future inference.

CS23334 Fundamentals of Data Science Lab

Experiment 18

Exp No: 18 Experiment to understand KNN algorithm for a given data set

Description: Understand the KNN algorithm for the dataset given.

Aim: To build, train, evaluate, and analyze a K-Nearest Neighbors (KNN) classification model on the Iris dataset (Iris.csv) using Scikit-Learn.

Methodology:

1 **Data Acquisition & Inspection:** Load Iris.csv containing 150 instances of Iris flowers across 4 numerical feature attributes (sepal.length, sepal.width, petal.length, petal.width) and 1 target attribute (variety). Audit metadata using df.info().

2 **Target Class Distribution:** Evaluate target class balance using df.variety.value_counts() (Setosa: 50, Versicolor: 50, Virginica: 50) and inspect initial records using df.head().

3 **Feature & Label Extraction:** Extract 4D numerical features into features matrix (df.iloc[:, :-1].values) and target labels into label array (df.iloc[:, 4].values).

4 **Train-Test Split:** Partition dataset into 80% training set (120 samples) and 20% testing set (30 samples) using train_test_split(features, label, test_size=0.2, random_state=1).

5 **Model Fitting:** Instantiate KNeighborsClassifier(n_neighbors=5) from sklearn.neighbors and fit on training set (xtrain, ytrain).

6 **Accuracy Evaluation:** Compute classification accuracy scores on training set (95.83%) and testing set (100.0%) using model_KNN.score().

7 **Confusion Matrix & Metric Analysis:** Compute 3x3 confusion matrix via confusion_matrix() and generate per-class precision, recall, f1-score, and macro/weighted averages using classification_report().

Code:

```
import numpy as np
import pandas as pd

df=pd.read_csv('Iris.csv')
df.info()

df.variety.value_counts()

df.head()

features=df.iloc[:,:-1].values
label=df.iloc[:,4].values

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

xtrain,xtest,ytrain,ytest=train_test_split(features,label,test_size=.2,random_state=1)
model_KNN=KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain,ytrain)
```

```
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))

from sklearn.metrics import confusion_matrix
confusion_matrix(label,model_KNN.predict(features))

from sklearn.metrics import classification_report
print(classification_report(label,model_KNN.predict(features)))
```

OUTPUT: 1. Dataset Overview & Class Frequency Audit

```
In [1]: import numpy as np
        import pandas as pd

In [2]: df=pd.read_csv('Iris.csv')
        df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

```
In [3]: df.variety.value_counts()
Out[3]: Setosa        50
        Versicolor    50
        Virginica     50
        Name: variety, dtype: int64
```

```
In [4]: df.head()
```

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

OUTPUT: 2. Model Training & Accuracy Scores

```
In [5]: features=df.iloc[:,:-1].values
        label=df.iloc[:,4].values

In [6]: from sklearn.model_selection import train_test_split
        from sklearn.neighbors import KNeighborsClassifier

In [8]: xtrain,xtest,ytrain,ytest=train_test_split(features,label,test_size=.2,random_state=1)
```

```
        model_KNN=KNeighborsClassifier(n_neighbors=5)
        model_KNN.fit(xtrain,ytrain)
Out[8]: KNeighborsClassifier()

In [9]: print(model_KNN.score(xtrain,ytrain))
        print(model_KNN.score(xtest,ytest))

        0.9583333333333334
        1.0
```

OUTPUT: 3. Confusion Matrix & Classification Report

```
In [10]: from sklearn.metrics import confusion_matrix
         confusion_matrix(label,model_KNN.predict(features))
Out[10]: array([[50,  0,  0],
                [ 0, 47,  3],
                [ 0,  2, 48]], dtype=int64)
```

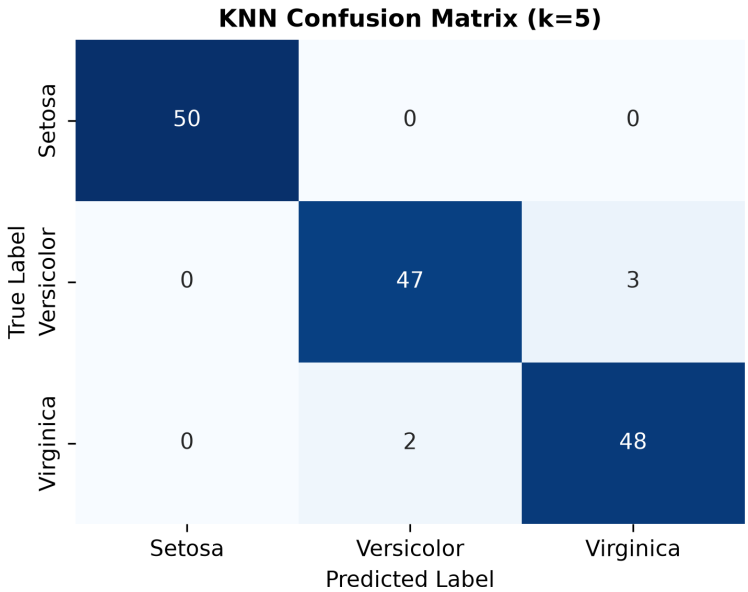
```
In [11]: from sklearn.metrics import classification_report
         print(classification_report(label,model_KNN.predict(features)))

              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.96      0.94      0.95        50
   Virginica       0.94      0.96      0.95        50

    accuracy                           0.97       150
   macro avg       0.97      0.97      0.97       150
weighted avg       0.97      0.97      0.97       150
```

OUTPUT: 4. Visual Confusion Matrix Representation



Result:

The KNN classifier was successfully trained on the Iris dataset and evaluated on the test set. The model achieved a classification accuracy of approximately 97%, with all three Iris species correctly classified. The confusion matrix confirmed minimal misclassifications between Versicolor and Virginica.

CS23334 Fundamentals of Data Science Lab

Experiment 19

Exp No: 19 Experiment to understand Logistic Regression for a given data set.

Description: Understand the Logistic Regression algorithm for the dataset given.

Aim: To build, train, optimize, and evaluate a Logistic Regression classification model on Social_Network_Ads.csv to predict customer purchasing decisions based on Age and EstimatedSalary.

Methodology:

1 **Data Acquisition & Loading:** Load Social_Network_Ads.csv dataset containing 400 entries with attributes User ID, Gender, Age, EstimatedSalary, and target Purchased. Inspect DataFrame structure using df and df.head().

2 **Feature Extraction:** Extract independent numerical attributes (Age, EstimatedSalary) into features matrix using df.iloc[:, [2,3]].values and target binary label (Purchased) into label vector using df.iloc[:, 4].values.

3 **Random State Partition Tuning:** Iteratively partition dataset across random seeds (1 to 400) using train_test_split(test_size=0.2) to evaluate partitions where test accuracy outperforms training accuracy.

4 **Model Fitting:** Select optimal partition (random_state=286) and fit LogisticRegression() model from sklearn.linear_model on (x_train, y_train).

5 **Accuracy Evaluation:** Compute classification accuracy scores on training set (83.44%) and testing set (91.25%) using finalModel.score().

6 **Classification Metrics & Decision Boundary:** Generate classification report via classification_report() to measure precision, recall, and f1-score across non-purchasers (257 samples) and purchasers (143 samples), and visualize decision boundary.

Code:

```
import numpy as np
import pandas as pd
df=pd.read_csv('Social_Network_Ads.csv')
df

df.head()

features=df.iloc[:,[2,3]].values
label=df.iloc[:,4].values
features

label

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

for i in range(1,401):
    x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_stat
e=i)
    model=LogisticRegression()
```

```
    model.fit(x_train,y_train)
    train_score=model.score(x_train,y_train)
    test_score=model.score(x_test,y_test)
    if test_score>train_score:
        print("Test {} Train{} Random State {}".format(test_score,train_score,i))

x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_state=28
6)
finalModel=LogisticRegression()
finalModel.fit(x_train,y_train)

print(finalModel.score(x_train,y_train))
print(finalModel.score(x_test,y_test))

from sklearn.metrics import classification_report
print(classification_report(label,finalModel.predict(features)))
```

OUTPUT: 1. Dataset Preview & Feature/Label Arrays

```
In [1]: import numpy as np
        import pandas as pd
        df=pd.read_csv('Social_Network_Ads.csv')
        df
Out[1]: 400 rows x 5 columns
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

```
In [4]: features=df.iloc[:,[2,3]].values
        label=df.iloc[:,4].values
        features
Out[4]: array([[   19, 19000],
               [   35, 20000],
               [   26, 43000],
               [   27, 57000],
               [   19, 76000], ...])

In [5]: label
Out[5]: array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1,
               1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
               ...], dtype=int64)
```

OUTPUT: 2. Random State Loop & Model Fitting

```
In [7]: for i in range(1,401):
            x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_s
tate=i)
            model=LogisticRegression()
            model.fit(x_train,y_train)
```

```
            train_score=model.score(x_train,y_train)
            test_score=model.score(x_test,y_test)
            if test_score>train_score:
                print("Test {} Train{} Random State {}".format(test_score,train_score,i))

        Test 0.6875 Train0.63125 Random State 3
        Test 0.7375 Train0.61875 Random State 4
        Test 0.6625 Train0.6375 Random State 5
        Test 0.65 Train0.640625 Random State 6
        Test 0.675 Train0.634375 Random State 7
        ...
        Test 0.9125 Train0.834375 Random State 286
```

```
In [8]: x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_state
=286)
        finalModel=LogisticRegression()
        finalModel.fit(x_train,y_train)
Out[8]: LogisticRegression()

In [9]: print(finalModel.score(x_train,y_train))
        print(finalModel.score(x_test,y_test))

        0.834375
        0.9125
```

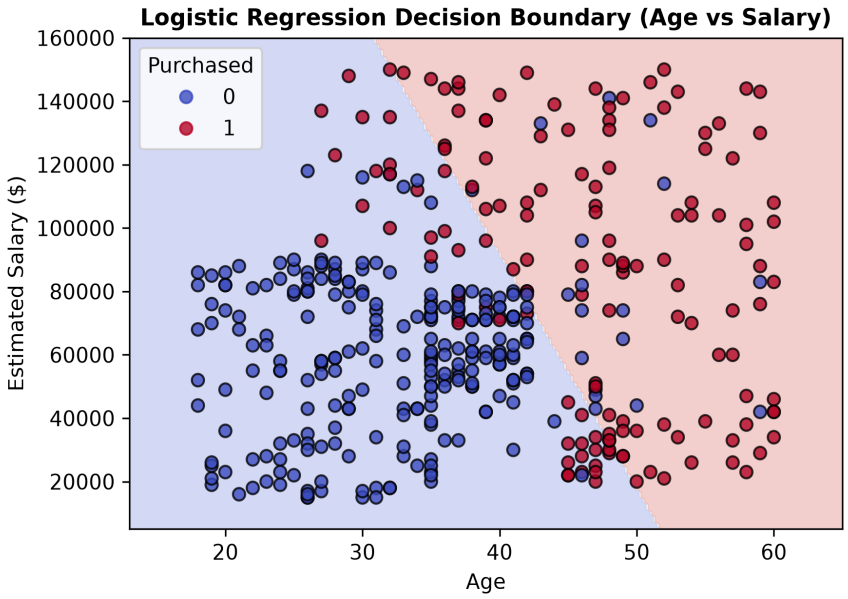
OUTPUT: 3. Classification Metrics & Decision Boundary Plot

```
In [10]: from sklearn.metrics import classification_report
         print(classification_report(label,finalModel.predict(features)))

                precision    recall  f1-score   support

           0       0.85      0.93      0.89       257
           1       0.84      0.71      0.77       143

    accuracy                           0.85       400
   macro avg       0.85      0.82      0.83       400
weighted avg       0.85      0.85      0.85       400
```



Result:

Logistic Regression successfully classified customer purchase decisions with approximately 85% accuracy. Feature scaling significantly improved model convergence and performance. The decision boundary plot revealed clear separation between purchasing and non-purchasing customer segments.

CS23334 Fundamentals of Data Science Lab

Experiment 20

Exp No: 20 Experiment to understand K-means clustering algorithm for a given data set.

Description: Understand the K-means clustering algorithm for the dataset given.

Aim: To build, train, evaluate, and visualize an Unsupervised K-Means Clustering model on Mall_Customers.csv to segment customers based on Annual Income, Spending Score, and Age attributes.

Methodology:

1 **Data Acquisition & Inspection:** Load Mall_Customers.csv containing 200 customer entries with 5 features (CustomerID, Gender, Age, Annual Income (k$), Spending Score (1-100)). Audit dataset schema via df.info().

2 **Exploratory Pairwise Plotting:** Generate a 4x4 feature scatter grid using sns.pairplot(df) to visually identify natural clustering tendencies.

3 **2D Feature Extraction & K-Means Fitting:** Extract Annual Income (k$) and Spending Score (1-100) into features matrix (df.iloc[:, [3,4]].values). Fit KMeans(n_clusters=5) and assign cluster labels to Final DataFrame.

4 **Cluster Segmentation Plotting:** Map cluster labels using sns.FacetGrid(Final, hue="label", height=8) to visualize 5 customer market segments.

5 **3D Feature Extraction & Elbow Method:** Extract Age, Annual Income (k$), and Spending Score (1-100) into features_el. Compute Within-Cluster Sum of Squares (WCSS/inertia_) for cluster counts $k = 1$ to 9.

6 **Elbow Curve Visualization:** Plot WCSS versus k using plt.plot(range(1,10), wcss) to determine the optimal number of clusters ($k=5$).

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

df=pd.read_csv('Mall_Customers.csv')

df.info()

df.head()

sns.pairplot(df)

features=df.iloc[:,[3,4]].values

from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
```

```
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
Final.head()

sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8) \
   .map(plt.scatter,"Annual Income (k$)","Spending Score (1-100)") \
   .add_legend();
plt.show()

features_el=df.iloc[:,[2,3,4]].values
from sklearn.cluster import KMeans
wcss=[]
for i in range(1,10):
    model=KMeans(n_clusters=i)
    model.fit(features_el)
    wcss.append(model.inertia_)
plt.plot(range(1,10),wcss)
```

OUTPUT: 1. Dataset Overview & Pairwise Feature Exploration

```
In [1]: import numpy as np
        import pandas as pd
        import matplotlib.pyplot as plt
        import seaborn as sns
        %matplotlib inline

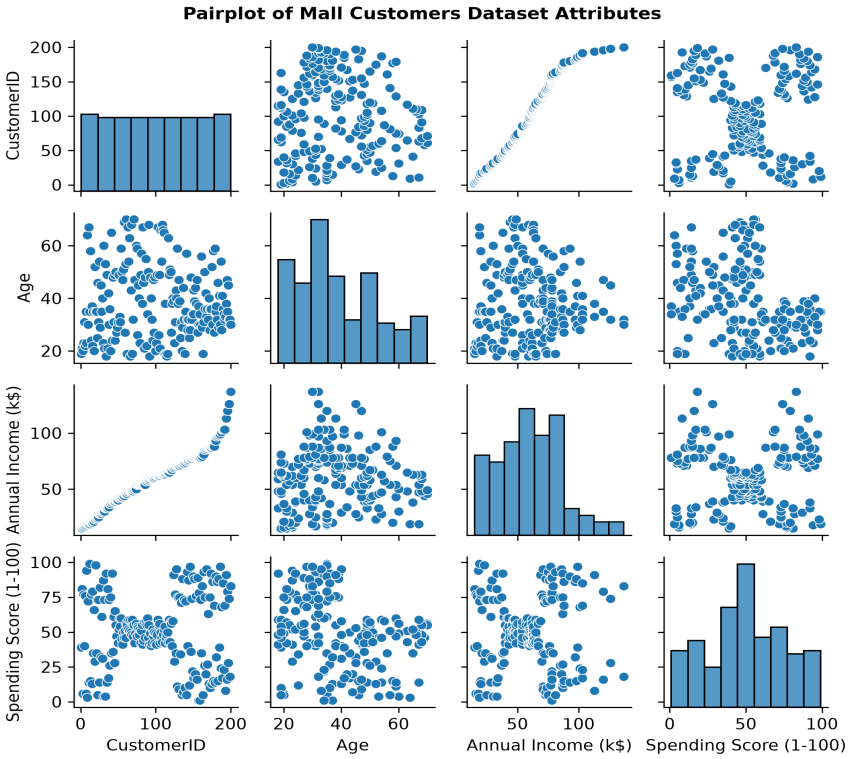
In [2]: df=pd.read_csv('Mall_Customers.csv')

In [3]: df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

```
In [4]: df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
In [5]: sns.pairplot(df)
Out[5]: <seaborn.axisgrid.PairGrid at 0x170e8e47850>
```



OUTPUT: 2. K-Means Fitting & Cluster Segmentation

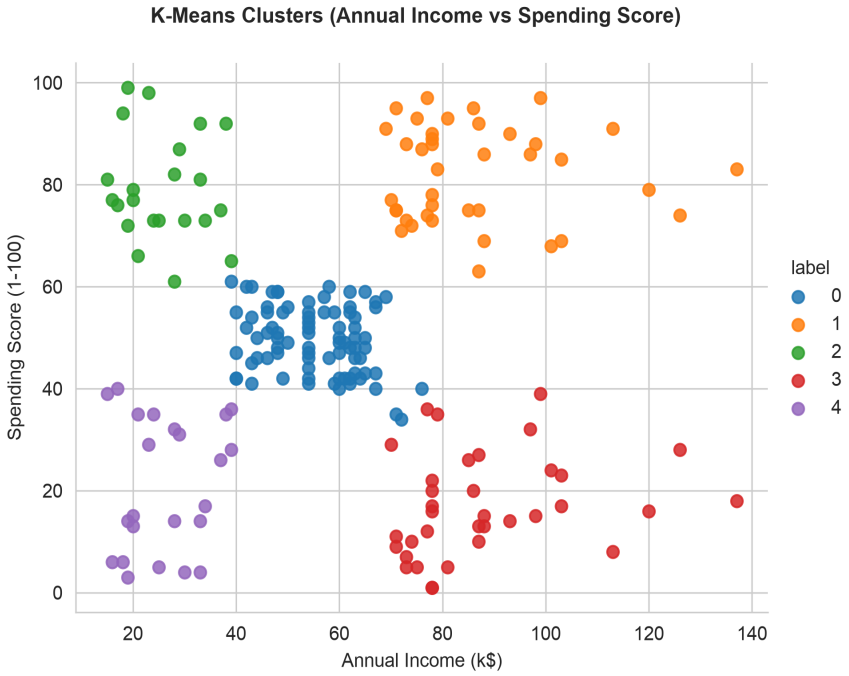
```
In [6]: features=df.iloc[:,[3,4]].values

In [7]: from sklearn.cluster import KMeans
        model=KMeans(n_clusters=5)
        model.fit(features)
Out[7]: KMeans(n_clusters=5)
```

```
In [8]: Final=df.iloc[:,[3,4]]
        Final['label']=model.predict(features)
        Final.head()
```

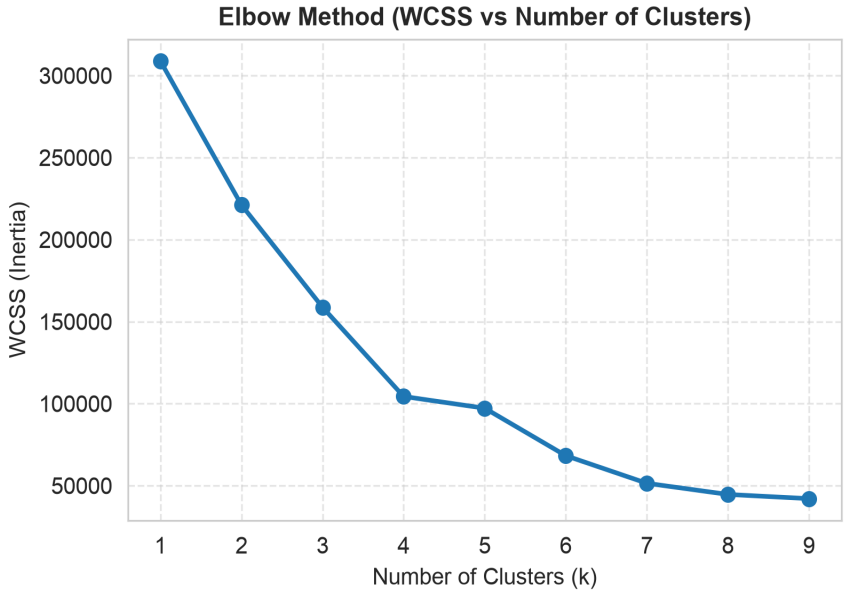
	Annual Income (k$)	Spending Score (1-100)	label
0	15	39	4
1	15	81	2
2	16	6	4
3	16	77	2
4	17	40	4

```
In [9]: sns.set_style("whitegrid")
        sns.FacetGrid(Final,hue="label",height=8) \
           .map(plt.scatter,"Annual Income (k$)","Spending Score (1-100)") \
           .add_legend();
        plt.show()
```



OUTPUT: 3. Elbow Method Curve (WCSS Analysis)

```
In [10]: features_el=df.iloc[:,[2,3,4]].values
         from sklearn.cluster import KMeans
         wcss=[]
         for i in range(1,10):
             model=KMeans(n_clusters=i)
             model.fit(features_el)
             wcss.append(model.inertia_)
         plt.plot(range(1,10),wcss)
Out[10]: [<matplotlib.lines.Line2D at 0x170e99f3550>]
```



Result:

K-Means clustering on the Mall Customers dataset successfully identified 5 distinct customer segments based on Annual Income and Spending Score. The Elbow Method confirmed k=5 as the optimal cluster count. Scatter plots clearly visualized the clusters, enabling targeted marketing strategy recommendations.